

LaStatale Blue
Team Reference Document

16/02/2023

CONTENTS

1.	
1.1.	Bashr
1.2.	Header
1.3.	Template Sho
1.4.	Utility
2.	Extra
2.1.	MinCostMaxFlow
2.2.	ModularLinearEquationSolver
2.3.	Delaunay
2.4.	Manacher
2.5.	Stablemp
2.6.	Totient
2.7.	Welzl
3.	Mathematics
3.1.	Fft
3.2.	Greatest Common Divisor
3.3.	Miller Rabin Test
3.4.	Ntt
4.	Data Structures
4.1.	Avl Tree
4.2.	Heavy Light Edge
4.3.	Heavy Light Node
4.4.	Pairing Heap
4.5.	Persistent Segment Tree
4.6.	Scapegoat
4.7.	Sparse Table
4.8.	Treap
5.	Generic
5.1.	Hornsat
5.2.	Introselect
5.3.	Sat
5.4.	Simplex
5.5.	Twosat
5.6.	Twosat Dewd
5.7.	Xorsat
6.	Graphs
6.1.	Bipartite Matching
6.2.	Edmonds Blossom
6.3.	Edmonds Blossom Weighted
6.4.	Eulerian
6.5.	Hopcroft Karp
6.6.	Hungarian
6.7.	Kosaraju Scc
6.8.	Min Vertex Cover Bipartited
6.9.	Tarjan Artpoints Bridges
6.10.	Tidal Flow
6.11.	Toposort
6.12.	Tsp
7.	Strings

7.1.	Aho Corasick
7.2.	Aho Corasick
7.3.	Cyk
7.4.	Dbf
7.5.	Dc3
7.6.	Rabin Karp
7.7.	Suffix Array
7.8.	Z Algorithm
8.	Computational Geometry
8.1.	Closest Pair
8.2.	Convex Hull
8.3.	Geocomp
8.4.	Poi
8.5.	Point In Polygon
8.6.	Polygon Area
8.7.	SegIsect
	1
1.1.	Bashr.
4	function check {
4	IFS=
4	s=""
5	cat \$1 while read l; do
5	s="\$s\$(echo \$l sed 's/\s//g')\n"
6	h=\$(echo -ne "\$s" md5sum)
8	echo "\${h:0:2} \$1"
9	done
9	}
11	alias ll='ls -alh'
11	
12	1.2. Header.
12	#include <bits/stdc++.h>
12	using namespace std;
13	template <class T> int size(const T& x) {
13	return x.size();
14	}
14	#define rep(i, a, b) for(__typeof(a) i = (a); i < (b); ++i)
14	#define iter(it, c)
15	for(__typeof((c).begin()) it = (c).begin();
15	it != (c).end(); ++it)
15	typedef pair<int, int> ii;
15	typedef vector<int> vi;
17	typedef vector<ii> vii;
17	typedef long long ll;
18	const int INF = ~(1 << 31);
19	
19	const double EPS = 1e-9;
19	const double pi = acos(-1);
19	typedef unsigned long long ull;
20	typedef vector<vi> vvi;
21	typedef vector<vii> vvii;
21	template <class T> T smod(T a, T b) {

21	return (a % b + b) % b;	//9e
21	}	//5a
22		
22	1.3. Template Sho.	
23	#include <bits/stdc++.h>	
23	using namespace std;	
23		
24	using ll = long long;	
24	using db = long double; // or double if tight TL	
24		
24	using pi = pair<int,int>;	
24	#define mp make_pair	
25	#define f first	
25	#define s second	
25		
25	#define tcT template<class T	
	tcT> using V = vector<T>;	
	tcT, size_t SZ> using AR = array<T,SZ>;	
	using vi = V<int>;	
	using vpi = V<pi>;	
	#define sz(x) int((x).size())	
	#define all(x) begin(x), end(x)	
//09	#define sor(x) sort(all(x))	
//02		
	#define rsz resize	
//7d	#define pb push_back	
//e9	#define ft front()	
//62	#define bk back()	
//f1		
//2a	#define FOR(i,a,b) for (int i = (a); i < (b); ++i)	
//76	#define FOR(i,a) FOR(i,0,a)	
//a2	#define ROF(i,a,b) for (int i = (b)-1; i >= (a); --i)	
//93	#define ROF(i,a) ROF(i,0,a)	
//f0	#define rep(a) FOR(_,a)	
	#define each(a,x) for (auto& a: x)	
//84	1.4. Utility.	
//16	#include <bits/stdc++.h>	//84
//71		//8e
//df	using namespace std;	//ca
//92		//b2
//88	template <typename T>	//bb
//bb	ostream& operator<<(ostream& o, const vector<T>& v) {	//ed
//2d	o << "[";	//ad
//99	for(size_t i = 0; i < v.size(); i++) {	//1d
//7e	o << v[i];	//f8
//d5	if(i != v.size() - 1) {	//97
//cf	o << ", ";	//47
//4a	}	//9b
//d9	}	//b7
//c8	o << "]";	//c4
//e2	return o;	//92
//a4	}	//27
//43		//83
//3b	template <typename T>	//b1
//47	ostream& operator<<(ostream& o,	//a4
//18	const unordered_set<T>& s) {	//87

```
o << "{";
auto it = s.begin();
for(size_t i = 0; i < s.size(); i++, ++it) {
    o << *it;
    if(i != s.size() - 1) {
        o << ", ";
    }
}
o << "}";
return o;
}

template <typename T>
vector<vector<T>>
transpose_adj(const vector<vector<T>>& adj) {
    T n_trans = T(0);
    for(const auto& v : adj) {
        n_trans =
            max(n_trans, *max_element(v.begin(), v.end()) + 1);
    }
    vector<vector<T>> adjT(n_trans);
    for(size_t i = 0; i < adj.size(); i++) {
        for(const auto& j : adj[i]) {
            adjT[j].push_back(i);
        }
    }
    return adjT;
}
```

2. EXTRA

2.1. MinCostMaxFlow.

```
//68
//e1
typedef vector<int> VI;
typedef vector<VI> VVI;
typedef long long L;
typedef vector<L> VL;
typedef vector<VL> VVL;
typedef pair<int, int> PII;
typedef vector<PII> VPII;

const L INF = numeric_limits<L>::max() / 4;

struct MinCostMaxFlow {
    int N;
    VVL cap, flow, cost;
    VI found;
    VL dist, pi, width;
    VPII dad;

    MinCostMaxFlow(int N)
        : N(N), cap(N, VL(N)), flow(N, VL(N)),
          cost(N, VL(N)), found(N), dist(N), pi(N), width(N),
          dad(N) {}

    void AddEdge(int from, int to, L cap, L cost) {
```

```
this->cap[from][to] = cap;
this->cost[from][to] = cost;
}

void Relax(int s, int k, L cap, L cost, int dir) {
    L val = dist[s] + pi[s] - pi[k] + cost;
    if(cap && val < dist[k]) {
        dist[k] = val;
        dad[k] = make_pair(s, dir);
        width[k] = min(cap, width[s]);
    }
}

L Dijkstra(int s, int t) {
    fill(found.begin(), found.end(), false);
    fill(dist.begin(), dist.end(), INF);
    fill(width.begin(), width.end(), 0);
    dist[s] = 0;
    width[s] = INF;

    while(s != -1) {
        int best = -1;
        found[s] = true;
        for(int k = 0; k < N; k++) {
            if(found[k])
                continue;
            Relax(s, k, cap[s][k] - flow[s][k], cost[s][k], 1);
            Relax(s, k, flow[k][s], -cost[k][s], -1);
            if(best == -1 || dist[k] < dist[best])
                best = k;
        }
        s = best;
    }

    for(int k = 0; k < N; k++)
        pi[k] = min(pi[k] + dist[k], INF);
    return width[t];
}

pair<L, L> GetMaxFlow(int s, int t) {
    L totflow = 0, totcost = 0;
    while(L amt = Dijkstra(s, t)) {
        totflow += amt;
        for(int x = t; x != s; x = dad[x].first) {
            if(dad[x].second == 1) {
                flow[dad[x].first][x] += amt;
                totcost += amt * cost[dad[x].first][x];
            } else {
                flow[x][dad[x].first] -= amt;
                totcost -= amt * cost[x][dad[x].first];
            }
        }
    }
    return make_pair(totflow, totcost);
}
```

```
2.2. ModularLinearEquationSolver.

#include "ExtendedEuclid.h"

vector<int> modular_linear_equation_solver(int a, int b,
                                           int n) {
    int x, y;
    vector<int> sol;
    int d = extended_euclid(a, n, x, y);
    if(!(b % d)) {
        x = mod(x % (b / d), n);
        for(int i = 0; i < d; ++i)
            sol.push_back(mod(x + i * (n / d), n));
    }
    return sol;
}

int mod_inverse(int a, int n) {
    int x, y;
    int d = extended_euclid(a, n, x, y);
    if(d > 1)
        return -1;
    return mod(x, n);
}

void linear_diophantine(int a, int b, int c, int& x,
                        int& y) {
    int d = gcd(a, b);
    if(c % d)
        x = y = -1;
    else {
        x = c / d * mod_inverse(a / d, b / d);
        y = (c - a * x) / b;
    }
}

2.3. Delaunay.

typedef double T;

struct triple {
    int i, j, k;
    triple() {}
    triple(int i, int j, int k) : i(i), j(j), k(k) {}
};

vector<triple> delaunayTriangulation(vector<T>& x,
                                    vector<T>& y) {
    int n = x.size();
    vector<T> z(n);
    vector<triple> ret;

    for(int i = 0; i < n; i++)
        z[i] = x[i] * x[i] + y[i] * y[i];
}
```

```
for(int i = 0; i < n - 2; i++) { //1b
    for(int j = i + 1; j < n; j++) { //5f
        for(int k = i + 1; k < n; k++) { //1d
            if(j == k) //2b
                continue; //9b
            double xn = (y[j] - y[i]) * (z[k] - z[i]) - //98
                (y[k] - y[i]) * (z[j] - z[i]); //fb
            double yn = (x[k] - x[i]) * (z[j] - z[i]) - //9c
                (x[j] - x[i]) * (z[k] - z[i]); //91
            double zn = (x[j] - x[i]) * (y[k] - y[i]) - //ef
                (x[k] - x[i]) * (y[j] - y[i]); //ce
            bool flag = zn < 0; //a3
            for(int m = 0; flag && m < n; m++) //f4
                flag = flag && //5d
                    ((x[m] - x[i]) * xn + (y[m] - y[i]) * yn + //b8
                     (z[m] - z[i]) * zn <= //e1
                     0); //f6
            if(flag) //55
                ret.push_back(triple(i, j, k)); //27
        } //a3
    } //3d
} //7f
return ret; //cf
} //9d

2.4. Manacher.

vector<int> fastLongestPalindromes(string str) { //68
    int i = 0, j, d, s, e, lLen, palLen = 0; //00
    vector<int> res; //e2
    while(i < str.length()) { //5e
        if(i > palLen && str[i - palLen - 1] == str[i]) { //42
            palLen += 2; //56
            i++; //74
            continue; //5e
        } //a8
        res.push_back(palLen); //a5
        s = res.size() - 2; //b1
        e = s - palLen; //2c
        bool b = true; //79
        for(j = s; j > e; j--) { //0c
            d = j - e - 1; //0d
            if(res[j] == d) { //7d
                palLen = d; //fc
                b = false; //36
                break; //88
            } //9d
            res.push_back(min(d, res[j])); //a1
        } //34
        if(b) { //39
            palLen = 1; //04
            i++; //e1
        } //3e
    } //d5
    res.push_back(palLen); //f0
    lLen = res.size(); //8d
} //58

//1b s = lLen - 2; //21
//5f e = s - (2 * str.length() + 1 - lLen); //d3
//1d for(i = s; i > e; i--) { //c8
//2b d = i - e - 1; //ae
//9b res.push_back(min(d, res[i])); //22
//98 } //35
//fb return res; //22
//9c } //72

2.5. Stablemp.

pair<vector<int>, vector<int>> //68
stable_marriage(vector<vector<int>>& madj, //a6
vector<vector<int>>& fpref) { //6b
    int n = madj.size(); //71
    vector<int> mpart(n, -1), fpart(n, -1); //bd
    vector<int> midx(n); //ab
    queue<int> mfree; //c1
    for(int i = 0; i < n; i++) { //d8
        mfree.push(i); //3b
    } //2c
    while(!mfree.empty()) { //cd
        int m = mfree.front(); //dd
        mfree.pop(); //b3
        int f = madj[m][midx[m]++]; //0d
        if(fpart[f] == -1) { //0e
            mpart[m] = f; //f5
            fpart[f] = m; //de
        } else if(fpref[f][m] < fpref[f][fpart[f]]) { //f4
            mpart[fpart[f]] = -1; //3b
            mfree.push(fpart[f]); //6a
            mpart[m] = f; //7e
            fpart[f] = m; //3a
        } else { //0b
            mfree.push(m); //0e
        } //91
    } //62
    return make_pair(mpart, fpart); //98
} //10

2.6. Totient.

#define MAX 10000000 //68
int phi[MAX]; //e0
bool pr[MAX]; //e5
void totient() { //a2
    for(int i = 0; i < MAX; i++) { //e9
        phi[i] = i; //05
        pr[i] = true; //5b
    } //96
    for(int i = 2; i < MAX; i++) //80
        if(pr[i]) { //87
            for(int j = i; j < MAX; j += i) { //02
                pr[j] = false; //d3
                phi[j] = phi[j] - (phi[j] / i); //56
            } //ca
        } //78
    } //70

} //7a
pr[i] = true; //04
} //85
} //88

2.7. Welzl.

struct point { //68
    double x; //52
    double y; //2b
}; //25
struct circle { //be
    double x; //4e
    double y; //5b
    double r; //24
    circle() {} //5b
    circle(double x, double y, double r) {} //ac
}; //90
circle b_md(vector<point> R) { //f4
    if(R.size() == 0) { //63
        return circle(0, 0, -1); //d3
    } else if(R.size() == 1) { //2d
        return circle(R[0].x, R[0].y, 0); //be
    } else if(R.size() == 2) { //4c
        return circle( //a4
            (R[0].x + R[1].x) / 2.0, (R[0].y + R[1].y) / 2.0, //83
            hypot(R[0].x - R[1].x, R[0].y - R[1].y) / 2.0); //b3
    } else { //c9
        double D = (R[0].x - R[2].x) * (R[1].y - R[2].y) - //bb
            (R[1].x - R[2].x) * (R[0].y - R[2].y); //87
        double p0 = (((R[0].x - R[2].x) * (R[0].x + R[2].x) + //d0
            (R[0].y - R[2].y) * (R[0].y + R[2].y)) / //83
            2 * (R[1].y - R[2].y) - //c6
            ((R[1].x - R[2].x) * (R[1].x + R[2].x) + //e9
            (R[1].y - R[2].y) * (R[1].y + R[2].y)) / //13
            2 * (R[0].y - R[2].y)) / //85
            D; //c7
        double p1 = (((R[1].x - R[2].x) * (R[1].x + R[2].x) + //6e
            (R[1].y - R[2].y) * (R[1].y + R[2].y)) / //7e
            2 * (R[0].x - R[2].x) - //19
            ((R[0].x - R[2].x) * (R[0].x + R[2].x) + //8c
            (R[0].y - R[2].y) * (R[0].y + R[2].y)) / //38
            2 * (R[1].x - R[2].x)) / //38
            D; //25
        return circle(p0, p1, hypot(R[0].x - p0, R[0].y - p1)); //57
    } //c8
} //14
circle b_minidisk(vector<point>& P, int i, //33
vector<point> R) { //9e
    if(i == P.size() || R.size() == 3) { //58
        return b_md(R); //bf
    } else { //a4
        circle D = b_minidisk(P, i + 1, R); //7f
        if(hypot(P[i].x - D.x, P[i].y - D.y) > D.r) { //01
            R.push_back(P[i]); //c2
        }
    }
}
```

```
D = b_minidisk(P, i + 1, R);
}
return D;
}
}

circle minidisk(vector<point> P) {
    random_shuffle(P.begin(), P.end());
    return b_minidisk(P, 0, vector<point>());
}
```

3. MATHEMATICS

```
3.1. Fft.

const int maxn = 400005;
const int g = 3, bigp = 479 * (1 << 21) + 1, x = 21;

size_t LA, LB;
int a[maxn], b[maxn], w[maxn];
size_t C, N, L;

int powc(int a, int b) {
    if(b == 0)
        return 1;

    int d = powc(a, b / 2);
    d = ((long long)d * d) % bigp;
    if(b & 1)
        d = ((long long)d * a) % bigp;
    return d;
}

void FFT(int x[], int K, bool inverse = false) {
    w[0] = 1;
    int G = powc(g, (bigp - 1) / K);

    for(int i = 1; i <= K; ++i) {
        w[i] = ((long long)w[i - 1] * G) % bigp;
    }
    for(int i = 0, j = 0; i < K; ++i) {
        if(i > j)
            swap(x[i], x[j]);
        for(int l = K >> 1; (j ^= 1) < 1; l >>= 1)
            ;
    }

    for(int i = 2; i <= K; i <= 1) {
        for(int j = 0; j < K; j += i) {
            for(int l = 0; l < (i >> 1); ++l) {
                int t = ((long long)x[j + 1 + (i >> 1)] *
                    w[inverse ? (K - (K / i) * l)
                        : (K / i) * l]) %
                    bigp;
                x[j + 1 + (i >> 1)] =
                    ((long long)x[j + 1] - t + bigp) % bigp;
                x[j + 1] += t;
                x[j + 1] %= bigp;
            }
        }
    }
}
```

```
}
}
}
if(inverse) {
    int r = powc(N, bigp - 2);
    for(size_t i = 0; i < N; ++i) {
        a[i] = ((long long)a[i] * r) % bigp;
    }
}

void calculate() {
    N = 1;
    C = LA + LB;
    L = 0;

    while(N <= C) {
        N *= 2;
        ++L;
    }

    for(size_t i = LA; i < N; ++i)
        a[i] = 0;
    for(size_t i = LB; i < N; ++i)
        b[i] = 0;

    FFT(a, N);
    FFT(b, N);

    for(size_t i = 0; i < N; ++i) {
        a[i] = ((long long)a[i] * b[i]) % bigp;
    }

    FFT(a, N, true);
}

3.2. Greatest Common Divisor.

template <typename T> T gcd(T a, T b, T& x, T& y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    T d = gcd(b, a % b, x, y);
    x -= a / b * y;
    swap(x, y);
    return d;
}

3.3. Miller Rabin Test.

template <class T> T mod_pow(T b, T e, T m) {
    T res = T(1);
    while(e) {
        if(e & T(1))
            res = smod(res * b, m);
        b = ((b * b) % m + m) % m, e >>= T(1);
    }
}
```

```
}
return res;
}

bool is_probable_prime(long long n, int k) {
    if(~n & 1)
        return n == 2;
    if(n <= 3)
        return n == 3;
    int s = 0;
    long long d = n - 1;
    while(~d & 1)
        d >>= 1, s++;
    while(k--) {
        long long a = (n - 3) * rand() / RAND_MAX + 2;
        long long x = mod_pow(a, d, n);
        if(x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for(int i = 0; i < s - 1; ++i) {
            x = (x * x) % n;
            if(x == 1)
                return false;
            if(x == n - 1) {
                ok = true;
                break;
            }
        }
        if(!ok)
            return false;
    }
    return true;
}

3.4. Ntt.

#include "utility.hpp"
#include <bits/stdc++.h>

using namespace std;

using u64 = unsigned long long int;

static vector<pair<u64, u64>> MAGIC = {
    {0ULL, 0ULL}, {3ULL, 2ULL},
    {5ULL, 2ULL}, {17ULL, 3ULL},
    {17ULL, 3ULL}, {97ULL, 5ULL},
    {193ULL, 5ULL}, {257ULL, 3ULL},
    {257ULL, 3ULL}, {7681ULL, 17ULL},
    {12289ULL, 11ULL}, {12289ULL, 11ULL},
    {12289ULL, 11ULL}, {40961ULL, 3ULL},
    {65537ULL, 3ULL}, {65537ULL, 3ULL},
    {65537ULL, 3ULL}, {786433ULL, 10ULL},
    {786433ULL, 10ULL}, {5767169ULL, 3ULL},
    {7340033ULL, 3ULL}, {23068673ULL, 3ULL},
    {104857601ULL, 3ULL}, {167772161ULL, 3ULL},
    {167772161ULL, 3ULL}, {167772161ULL, 3ULL},
}
```

<pre>{469762049ULL, 3ULL}, {2013265921ULL, 31ULL}, {3221225473ULL, 5ULL}, {3221225473ULL, 5ULL}, {3221225473ULL, 5ULL}};</pre>	<pre>//df //85 //f2 //c6 //7d //5a //a7 //b3 //c8 //18 //c1 //2e //a5 //38 //b5 //99 //2c //67 //71 //d9 //57 //b7 //ee //f7 //1a //15 //18 //8e //35 //6a //6a //39 //59 //24 //93 //06 //06 //5b //13 //f8 //68 //80 //6d //0a //d0 //d6 //69 //eb //3a //4d //c9 //68 //18 //2f //85 //d6</pre>	<pre>u64 p = pri.first; u64 g = inv ? fastexp(pri.second, p - 2, p) : pri.second; u64 root = fastexp(g, (p - 1) / n, p); auto nlog = __builtin_ctzll(n); vector<u64> radix(nlog); for(u64 i = 0; i < n; i++) { u64 rev = bit_reverse(i) >> (64 - nlog); if(i < rev) { swap(v[i], v[rev]); } } radix[0] = root; for(u64 i = 1; i < nlog; i++) { radix[i] = (radix[i - 1] * radix[i - 1]) % p; } for(u64 s = 1; s <= nlog; s++) { u64 r = nlog - s, w = 1 << s; for(u64 i = 0; i < n; i += w) { u64 tf = 1; for(u64 j = i; j < i + w / 2; j++, tf = (tf * radix[r]) % p) { u64 t = (tf * v[j + w / 2]) % p; u64 u = v[j]; v[j] = (u + t) % p; v[j + w / 2] = (u + (p - t)) % p; } } } if(inv) { u64 f = fastexp(n, p - 2, p); for(auto& x : v) x = (x * f) % p; } vector<u64> mul(vector<u64>& a, vector<u64>& b, u64 base = 2) { assert(a.size() == b.size()); u64 max_el = base - 1; ntt(a, false, max_el); ntt(b, false, max_el); auto pri = sel_magic(a.size(), max_el); vector<u64> c(a.size()); for(size_t i = 0; i < c.size(); ++i) { c[i] = (a[i] * b[i]) % pri.first; } ntt(c, true, max_el); return c; }</pre>	<pre>//10 //63 //13 //63 //3c //c7 //f5 //e5 //60 //7d //26 //54 //13 //f1 //55 //44 //ee //15 //d0 //56 //cb //75 //ed //fd //75 //f7 //0d //57 //4b //bd //70 //f0 //12 //bc //61 //13 //33 //85 //72 //99 //d9 //e3 //c4 //a7 //72 //77 //66 //88 //53 //da //cd //ab</pre>	<pre>#define AVL_MULTISET 0 template <class T> struct avl_tree { struct node { T item; node *p, *l, *r; int size, height; node(const T& _item, node* _p = NULL) : item(_item), p(_p), l(NULL), r(NULL), size(1), height(0) {} }; avl_tree() : root(NULL) {} node* root; inline int sz(node* n) const { return n ? n->size : 0; } inline int height(node* n) const { return n ? n->height : -1; } inline bool left_heavy(node* n) const { return n && height(n->l) > height(n->r); } inline bool right_heavy(node* n) const { return n && height(n->r) > height(n->l); } inline bool too_heavy(node* n) const { return n && abs(height(n->l) - height(n->r)) > 1; } void delete_tree(node* n) { if(n) { delete_tree(n->l), delete_tree(n->r); delete n; } } node*& parent_leg(node* n) { if(!n->p) return root; if(n->p->l == n) return n->p->l; if(n->p->r == n) return n->p->r; assert(false); } void augment(node* n) { if(!n) return; n->size = 1 + sz(n->l) + sz(n->r); n->height = 1 + max(height(n->l), height(n->r)); } #define rotate(l, r) node* l = n->l; l->p = n->p; parent_leg(n) = l; n->l = l->r; if(l->r) l->r->p = n; l->r = n, n->p = l; augment(n), augment(l) void left_rotate(node* n) { rotate(r, l); }</pre>	<pre>//b5 //27 //15 //bf //71 //99 //68 //da //02 //c3 //1d //0d //cc //07 //92 //7e //22 //d9 //20 //f7 //c8 //83 //08 //7e //a1 //00 //7c //96 //19 //52 //4a //f0 //44 //44 //ed //55 //77 //dc //e4 //56 //c4 //21 //20 //0b //b0 //9f //24 //11 //19 //99 //01 //9f //60 //ec //eb //8e</pre>
--	--	--	--	---	--

4. DATA STRUCTURES

4.1. Avl Tree.

```
void right_rotate(node* n) { rotate(l, r); } //ab
void fix(node* n) { //fc
    while(n) { //be
        augment(n); //b3
        if(too_heavy(n)) { //54
            if(left_heavy(n) && right_heavy(n->l)) //fe
                left_rotate(n->l); //0a
            else if(right_heavy(n) && left_heavy(n->r)) //ec
                right_rotate(n->r); //4f
            if(left_heavy(n)) //75
                right_rotate(n); //54
            else //02
                left_rotate(n); //be
            n = n->p; //00
        } //27
        n = n->p; //6d
    } //23
} //bb
inline int size() const { return sz(root); } //41
node* find(const T& item) const { //1f
    node* cur = root; //6f
    while(cur) { //8e
        if(cur->item < item) //36
            cur = cur->r; //34
        else if(item < cur->item) //e4
            cur = cur->l; //bb
        else //0a
            break; //85
    } //51
    return cur; //1d
} //f7
node* insert(const T& item) { //cc
    node *prev = NULL, **cur = &root; //81
    while(*cur) { //12
        prev = *cur; //e9
        if((*cur)->item < item) //84
            cur = &((*cur)->r); //9e
        #if AVL_MULTISSET //0f
        else //3e
            cur = &((*cur)->l); //d6
        #else //e2
        else if(item < (*cur)->item) //30
            cur = &((*cur)->l); //e8
        else //1f
            return *cur; //9a
    } //a9
    #endif //b3
    node* n = new node(item, prev); //0a
    *cur = n, fix(n); //af
    return n; //db
} //c8
void erase(const T& item) { erase(find(item)); } //e5
void erase(node* n, bool free = true) { //d7
    if(!n) //a3
        return; //02
    if(!n->l && n->r) //43
        parent_leg(n) = n->r, n->r->p = n->p; //de
    else if(n->l && !n->r) //62
        parent_leg(n) = n->l, n->l->p = n->p; //b2
    else if(n->l && n->r) { //3d
        node* s = successor(n); //ac
        erase(s, false); //f7
        s->p = n->p, s->l = n->l, s->r = n->r; //6e
        if(n->l) //3d
            n->l->p = s; //52
        if(n->r) //36
            n->r->p = s; //d7
        parent_leg(n) = s, fix(s); //6e
        return; //92
    } else //cd
        parent_leg(n) = NULL; //72
    fix(n->p), n->p = n->l = n->r = NULL; //34
    if(free) //a1
        delete n; //05
    } //8d
} //0f
node* successor(node* n) const { //e7
    if(!n) //92
        return NULL; //18
    if(n->r) //cc
        return nth(0, n->r); //aa
    node* p = n->p; //0c
    while(p && p->r == n) //6d
        n = p, p = p->p; //31
    return p; //c8
} //1d
node* predecessor(node* n) const { //eb
    if(!n) //59
        return NULL; //de
    if(n->l) //b5
        return nth(n->l->size - 1, n->l); //74
    node* p = n->p; //dc
    while(p && p->l == n) //64
        n = p, p = p->p; //35
    return p; //15
} //a9
node* nth(int n, node* cur = NULL) const { //8e
    if(!cur) //44
        cur = root; //04
    while(cur) { //0c
        if(n < sz(cur->l)) //6e
            cur = cur->l; //6a
        else if(n > sz(cur->l)) //b4
            n -= sz(cur->l) + 1, cur = cur->r; //6c
        else //e3
            break; //26
    } //ac
    return cur; //f4
} //b0
int count_less(node* cur) { //29
    int sum = sz(cur->l); //e8
    while(cur) { //24
        if(cur->p && cur->p->r == cur) //80
            sum += 1 + sz(cur->p->l); //b3
        cur = cur->p; //aa
    } //34
    return sum; //98
} //36
void clear() { delete_tree(root), root = NULL; } //f0
}; //b7

4.2. Heavy Light Edge. //86
#define root 0 //3e
#define N 10100 //12
#define LN 14 //71
vector<int> adj[N], costs[N], indexx[N]; //0e
int baseArray[N], ptr; //d4
int chainNo, chainInd[N], chainHead[N], posInBase[N]; //04
int depth[N], pa[LN][N], otherEnd[N], subsize[N]; //db
int st[N * 6], qt[N * 6]; //ab
void make_tree(int cur, int s, int e) { //fd
    if(s == e - 1) { //48
        st[cur] = baseArray[s]; //6e
        return; //91
    } //11
    int c1 = (cur << 1), c2 = c1 | 1, m = (s + e) >> 1; //f2
    make_tree(c1, s, m); //c3
    make_tree(c2, m, e); //2f
    st[cur] = st[c1] > st[c2] ? st[c1] : st[c2]; //d7
} //62
void update_tree(int cur, int s, int e, int x, int val) { //52
    if(s > x || e <= x) //cb
        return; //89
    if(s == x && s == e - 1) { //68
        st[cur] = val; //cc
        return; //f5
    } //b0
    int c1 = (cur << 1), c2 = c1 | 1, m = (s + e) >> 1; //8f
    update_tree(c1, s, m, x, val); //b2
    update_tree(c2, m, e, x, val); //18
    st[cur] = st[c1] > st[c2] ? st[c1] : st[c2]; //90
} //e1
/// max value in range [S,E] //45
void query_tree(int cur, int s, int e, int S, int E) { //e3
    if(s >= E || e <= S) { //0a
        qt[cur] = -1; //ea
        return; //41
    } //83
    if(s >= S && e <= E) { //d9
        qt[cur] = st[cur]; //19
        return; //bb
    } //5c
    int c1 = (cur << 1), c2 = c1 | 1, m = (s + e) >> 1; //5b
    query_tree(c1, s, m, S, E); //ef
    query_tree(c2, m, e, S, E); //a9
}
```

```

}
qt[cur] = qt[c1] > qt[c2] ? qt[c1] : qt[c2];
}

/*
 * query_up:
 * It takes two nodes u and v, condition is that v is an
 * ancestor of u. We query the chain in which u is present
 * till chain head, then move to next chain up. We do that
 * way till u and v are in the same chain, we query for
 * that part of chain and break
 */
int query_up(int u, int v) {
    if(u == v)
        return 0;
    int uchain, vchain = chainInd[v], ans = -1;
    while(1) {
        uchain = chainInd[u];
        if(uchain == vchain) {
            if(u == v)
                break;
            query_tree(1, 0, ptr, posInBase[v] + 1,
                       posInBase[u] + 1);
            if(qt[1] > ans)
                ans = qt[1];
            break;
        }
        query_tree(1, 0, ptr, posInBase[chainHead[uchain]],
                   posInBase[u] + 1);
        if(qt[1] > ans)
            ans = qt[1];
        u = chainHead[uchain];
        u = pa[0][u];
    }
    return ans;
}

int LCA(int u, int v) {
    if(depth[u] < depth[v])
        swap(u, v);
    int diff = depth[u] - depth[v];
    for(int i = 0; i < LN; i++)
        if((diff >> i) & 1)
            u = pa[i][u];
    if(u == v)
        return u;
    for(int i = LN - 1; i >= 0; i--)
        if(pa[i][u] != pa[i][v]) {
            u = pa[i][u];
            v = pa[i][v];
        }
    return pa[0][u];
}

void query(int u, int v) {
    int lca = LCA(u, v);
    int ans = query_up(u, lca);
    int temp = query_up(v, lca);
    if(temp > ans)
        ans = temp;
    printf("%d\n", ans);
}

void change(int i, int val) {
    int u = otherEnd[i];
    update_tree(1, 0, ptr, posInBase[u], val);
}

/*
 * Actual HL-Decomposition part
 * Initially all entries of chainHead[] are set to -1.
 * So when ever a new chain is started, chain head is
 * correctly assigned. As we add a new node to chain, we
 * will note its position in the baseArray. In the first
 * for loop we find the child node which has maximum
 * sub-tree size. The following if condition is failed for
 * leaf nodes. When the if condition passes, we expand the
 * chain to special child. In the second for loop we
 * recursively call the function on all normal nodes.
 * chainNo++ ensures that we are creating a new chain for
 * each normal child.
 */
void HLD(int curNode, int cost, int prev) {
    if(chainHead[chainNo] == -1) {
        chainHead[chainNo] = curNode;
    }
    chainInd[curNode] = chainNo;
    posInBase[curNode] =
        ptr;
    baseArray[ptr++] = cost;

    int sc = -1, ncost;
    for(int i = 0; i < adj[curNode].size(); i++)
        if(adj[curNode][i] != prev) {
            if(sc == -1 ||
               subsize[sc] < subsize[adj[curNode][i]]) {
                sc = adj[curNode][i];
                ncost = costs[curNode][i];
            }
        }
    if(sc != -1) {
        HLD(sc, ncost, curNode);
    }
    for(int i = 0; i < adj[curNode].size(); i++)
        if(adj[curNode][i] != prev) {
            if(sc != adj[curNode][i]) {
                chainNo++;
                HLD(adj[curNode][i], costs[curNode][i], curNode);
            }
        }
}

void dfs(int cur, int prev, int _depth = 0) {
    pa[0][cur] = prev;
    depth[cur] = _depth;
    subsize[cur] = 1;
    for(int i = 0; i < adj[cur].size(); i++)
        if(adj[cur][i] != prev) {
            otherEnd[indexx[cur][i]] = adj[cur][i];
            dfs(adj[cur][i], cur, _depth + 1);
            subsize[cur] += subsize[adj[cur][i]];
        }
}

int main() {
    int t;
    scanf("%d", &t);
    while(t--) {
        ptr = 0;
        int n;
        scanf("%d", &n);
        for(int i = 0; i < n; i++) {
            adj[i].clear();
            costs[i].clear();
            indxx[i].clear();
            chainHead[i] = -1;
            for(int j = 0; j < LN; j++)
                pa[j][i] = -1;
        }
        for(int i = 1; i < n; i++) {
            int u, v, c;
            scanf("%d %d %d", &u, &v, &c);
            u--;
            v--;
            adj[u].push_back(v);
            costs[u].push_back(c);
            indxx[u].push_back(i - 1);
            adj[v].push_back(u);
            costs[v].push_back(c);
            indxx[v].push_back(i - 1);
        }

        chainNo = 0;
        dfs(root, -1);
        HLD(root, -1, -1);
        make_tree(1, 0, ptr);

        for(int i = 1; i < LN; i++)
            for(int j = 0; j < n; j++)
                if(pa[i - 1][j] != -1)
                    pa[i][j] = pa[i - 1][pa[i - 1][j]];

        while(1) {
            char s[100];
            scanf("%s", s);
            if(s[0] == 'D') {
                break;
            }
        }
    }
}

```

```

    } //20
    int a, b; //69
    scanf("%d %d", &a, &b); //5d
    if(s[0] == 'Q') { //94
        query(a - 1, b - 1); //a6
    } else { //09
        change(a - 1, b); //92
    } //db
} //7e
} //df
} //2f

```

4.3. Heavy Light Node.

```

template <typename T, long long nullvalue, T merge(T, T)> //68
class heavy_light_decompositor { //76
    struct segment_tree { //6f
        static constexpr int left(int i) { return i * 2 + 1; } //4e
        static constexpr int right(int i) { return i * 2 + 2; } //d7
        static constexpr int root(int i) { //6d
            return (i - 1) / 2; //59
        } //e3
    } //ba
    vector<T> v, u; //56

    void update_node(int i) { //8f
        v[i] = merge(v[left(i)], v[right(i)]); //0d
    } //b8

    void lazyupdate(size_t i) { //a0
        v[i] += u[i]; //1d
        if(i < v.size() / 2) { //48
            u[right(i)] += u[i] / 2; //73
            u[left(i)] += u[i] / 2; //15
        } //53
        u[i] = 0; //72
    } //95

    void build(const vector<T>& source) { //0e
        int l; //b4
        for(l = 1; l << l < (int)source.size(); l++) //9d
            ; //ea
        v.resize((l << (l + 1)) - 1, nullvalue); //65
        u.resize(v.size(), 0); //58
        for(size_t i = 0; i < source.size(); i++) //54
            v[i + v.size() / 2] = source[i]; //76
        for(int i = v.size() / 2 - 1; i >= 0; i--) //61
            v[i] = merge(v[left(i)], v[right(i)]); //f1
    } //83

    int a, b; //aa
    T query_(int i, int l, int r) { //fd
        lazyupdate(i); //5c
        if(l >= a && r <= b) //b3
            return v[i]; //99
        if(r < a || l > b) //76
            return nullvalue; //20
        int m = (l + r) / 2; //2e
        return merge(query_(left(i), l, m), //2e
            query_(right(i), m + 1, r)); //f8
    }
}

```

```

    return nullvalue; //54
    int m = (l + r) / 2; //37
    return merge(query_(left(i), l, m), //bd
        query_(right(i), m + 1, r)); //58
} //8c

T query(int qa, int qb) { //c8
    if(qa > qb) //3e
        return nullvalue; //fa
    a = qa; //aa
    b = qb; //7c
    return query_(0, 0, v.size() / 2); //49
} //35

T off; //fc
void update_query(int i, int l, int r) { //d0
    lazyupdate(i); //cf
    if(l >= a && r <= b) { //c5
        int x = off * (r - l + 1); //e8
        v[i] += x; //98
        if(l != r) { //f0
            u[left(i)] += x / 2; //ab
            u[right(i)] += x / 2; //f5
        } //df
        return; //5f
    } //77
    if(r < a || l > b) //13
        return; //c7
    int m = (l + r) / 2; //ff
    update_query(left(i), l, m); //e8
    update_query(right(i), m + 1, r); //64
    update_node(i); //e4
} //0f

void range_add(int qa, int qb, T k) { //9e
    a = qa, b = qb, off = k; //ec
    update_query(0, 0, v.size() / 2); //20
} //6d

}; //45

struct node; //a7
struct edge { //a2
    node* to; //23
    T w; //63
}; //9e

struct chain : segment_tree { //db
    node* up; //2e
    int level; //1c
    vector<T> source; //c1
    void build() { segment_tree::build(source); } //52
    chain(node* up, int level) : up(up), level(level) { //3c
    } //d9
}; //a9

struct node { //9b
    int level; //1d
    vector<T> source; //bf
    void build() { segment_tree::build(source); } //7f
    chain(node* up, int level) : up(up), level(level) { //37
    } //03
}; //b9

```

```

chain* ch; //8c
int index; //f1
vector<edge> e; //16
int size = 1; //bd
int size_dfs(node* from) { //bc
    for(edge& x : e) //80
        if(x.to != from) //ff
            size += x.to->size_dfs(this); //7a
    return size; //d5
} //f1

edge* get_heavy(node* from) { //b0
    edge* best = 0; //67
    for(edge& x : e) //65
        if(x.to != from) //2d
            if(best == 0 || x.to->size > best->to->size) //59
                best = &x; //55
    return best; //d7
} //09

void build_chain(node* from, chain* c) { //ca
    index = c->source.size() - //30
        1; //bb
    ch = c; //2c
    if(size == 1) //fa
        return c->build(); //6e
    edge* h = get_heavy(from); //ac
    c->source.push_back(h->w); //52
    h->to->build_chain(this, c); //70
    for(edge& x : e) //d8
        if(&x != h && x.to != from) { //1a
            chain* next = new chain(this, c->level + 1); //30
            next->source.push_back(x.w); //7f
            x.to->build_chain(this, next); //bd
        } //27
    } //7b
}; //26

struct hld : vector<node> { //05
    T query(int qa, int qb) { //5f
        node* a = this->data() + qa; //a2
        node* b = this->data() + qb; //55
        T ans = 0; //15
        if(a->ch->level < b->ch->level) //a7
            swap(a, b); //62
        while(a->ch->level > b->ch->level) { //ca
            ans = merge(ans, a->ch->query(0, a->index)); //63
            a = a->ch->up; //41
        } //d4
        while(a->ch != b->ch) { //13
            ans = merge(merge(ans, a->ch->query(0, a->index)), //7b
                b->ch->query(0, b->index)); //52
            a = a->ch->up; //4f
            b = b->ch->up; //92
        } //0f
        if(a->index > b->index) //a6

```

<pre>swap(a, b); ans = merge(ans, a->ch->query(a->index + 1, b->index)); return ans; } void update_path(int qa, int qb, T k) { node* a = this->data() + qa; node* b = this->data() + qb; if(a->ch->level < b->ch->level) swap(a, b); while(a->ch->level > b->ch->level) { a->ch->range_add(0, a->index, k); a = a->ch->up; } while(a->ch != b->ch) { a->ch->range_add(0, a->index, k); b->ch->range_add(0, b->index, k); a = a->ch->up; b = b->ch->up; } if(a->index > b->index) swap(a, b); a->ch->range_add(a->index + 1, b->index, k); } void setup() { this->front().size_dfs(nullptr); chain* c = new chain(0, 0); c->source.push_back(0); this->front().build_chain(0, c); } }; hld g; public: heavy_light_decompositor(vector<tuple<int, int, T>> tree) { g.resize(tree.size() + 1); for(auto t : tree) { int a, b; T c; tie(a, b, c) = t; g[a].e.push_back({&g[b], c}); g[b].e.push_back({&g[a], c}); } g.setup(); } T query(int a, int b) { return g.query(a, b); } void update(int a, int b, T k) { g.update_path(a, b, k); } };</pre>	<pre>//f9 //d9 //da //79 //9d //69 //b6 //a9 //45 //ab //e8 //24 //66 //2d //03 //12 //dd //44 //a0 //9b //9e //70 //b7 //c0 //33 //5b //62 //6d //b0 //76 //2c //e2 //2e //85 //6a //b8 //1f //5e //41 //94 //4f //57 //aa //b6 //77 //0b //f2 //89 //dc //58 //7e</pre>	<pre>pairing_heap<T>* next; pairing_heap<T>* prev; pairing_heap<T>* child; static pairing_heap<T>* merge_pairs(pairing_heap<T>* a) { if(!a) return nullptr; if(!a->next) return a; pairing_heap<T>* other = a->next->next; pairing_heap<T>* b = a->next; a->next = b->next = nullptr; b->prev = nullptr; if(other) other->prev = nullptr; return merge(merge(a, b), merge_pairs(other)); } public: pairing_heap(const T& _value = T(), pairing_heap<T>* _next = nullptr, pairing_heap<T>* _prev = nullptr, pairing_heap<T>* _child = nullptr) { value = _value; next = _next; prev = _prev; child = _child; } static const T& top(const pairing_heap<T>* a) { assert(a); return a->value; } static pairing_heap<T>* merge(pairing_heap<T>* a, pairing_heap<T>* b) { if(!a) return b; if(!b) return a; pairing_heap<T>* upper = b; pairing_heap<T>* lower = a; if(top(a) < top(b)) { upper = a; lower = b; } assert(!lower->next); lower->next = upper->child; lower->prev = upper; if(upper->child) { upper->child->prev = lower; } upper->child = lower; return upper; }</pre>	<pre>//16 //6e //ef //33 //64 //4c //0a //4d //75 //70 //34 //a0 //32 //88 //0d //5d //17 //91 //5e //3d //46 //16 //5f //a7 //26 //2a //cd //98 //2a //a1 //3b //0c //8e //1c //94 //f2 //a4 //98 //7e //26 //98 //c9 //e4 //59 //12 //39 //f5 //c1 //1d //9a //2f //79 //10 //3e //59 //f0</pre>	<pre>static pairing_heap<T>* insert(pairing_heap<T>* a, const T& v) { return merge(a, new pairing_heap<T>(v)); } static pairing_heap<T>* remove_top(pairing_heap<T>* a) { if(!a) return nullptr; pairing_heap<T>* child = a->child; if(child) child->prev = nullptr; delete a; return merge_pairs(child); } static pairing_heap<T>* remove_top_nodelete(pairing_heap<T>* a) { if(!a) return nullptr; pairing_heap<T>* child = a->child; if(child) child->prev = nullptr; a->next = a->prev = a->child = nullptr; return merge_pairs(child); } static pairing_heap<T>* decrease_key(pairing_heap<T>* root, pairing_heap<T>* node, const T& new_value) { assert(!(top(node) < new_value)); node->value = new_value; if(!node->prev) { assert(node == root); return node; } if(node->prev->child == node) { if(!(top(node) < top(node->prev))) return root; node->prev->child = node->next; } else { node->prev->next = node->next; } if(node->next) node->next->prev = node->prev; node->prev = nullptr; node->next = nullptr; return merge(node, root); };</pre>	<pre>//90 //31 //8e //80 //f6 //19 //42 //81 //54 //c9 //03 //c6 //2b //a9 //e6 //b5 //b3 //82 //52 //2d //3a //be //97 //91 //e7 //fb //b7 //84 //a6 //18 //f4 //78 //8e //dd //bd //a6 //10 //16 //3b //71 //81 //8a //f6 //41 //60 //a6 //e6 //20</pre>	<pre>4.5. Persistent Segment Tree. int segcnt = 0; struct segment { int l, r, lid, rid, sum; } segs[2000000]; int build(int l, int r) {</pre>	<pre>//cf //68 //fc //dd //2b</pre>
<pre>4.4. Pairing Heap. template <typename T> class pairing_heap { protected: T value;</pre>	<pre>//7f //0f //81</pre>	<pre>}</pre>					

```

//f8 void reset(const T& v = T()) {
//9d tree_size = 1;
//2d value = v;
//12 left_child = -1;
//7b right_child = -1;
//87 parent = -1;
//dd }
//17 _scapegoat_node(const T& v) { reset(v); }
//1c
//ee public:
//34 friend class scapegoat<T>;
//e1 };
//1e
//1c template <typename T> class scapegoat {
//e8 protected:
//38 double a;
//36 std::vector<_scapegoat_node<T>> nodes;
//50 std::queue<int> unused_nodes;
//ef int root_node;
//c5 unsigned max_size;
//01 int _find(const T& v, int node) {
//65 if(nodes[node].value == v)
//4e return node;
//31 if(v < nodes[node].value) {
//5c if(nodes[node].left_child == -1)
//3f return -1;
//2b return _find(v, nodes[node].left_child);
//da } else {
//f0 if(nodes[node].right_child == -1)
//b4 return -1;
//86 return _find(v, nodes[node].right_child);
//ce }
//c0 }
//e9 int _get_node_size(int node) {
//b8 if(node == -1)
//ac return 0;
return nodes[node].tree_size;
}
int _find_scapegoat(int node) {
if(node == root_node)
return node;
bool e1 = _get_node_size(nodes[node].left_child) <=
a * _get_node_size(node);
bool e2 = _get_node_size(nodes[node].right_child) <=
a * _get_node_size(node);
if(!(e1 && e2))
return node;
return _find_scapegoat(nodes[node].parent);
}
int _get_free_node(const T& v = T()) {
if(!unused_nodes.empty()) {
int tmp = unused_nodes.front();
unused_nodes.pop();
nodes[tmp].reset(v);
return tmp;
} else {
void push_back(_scapegoat_node<T>(v));
return nodes.size() - 1;
}
}
void _inorder(int node, std::vector<int>& v) {
if(node == -1)
return;
_inorder(nodes[node].left_child, v);
v.push_back(node);
_inorder(nodes[node].right_child, v);
}
void _rr(int s, int e, std::vector<int>& r, int parent,
bool left) {
if(s == e)
return;
int m = (s + e) / 2;
nodes[r[m]].left_child = -1;
nodes[r[m]].right_child = -1;
nodes[r[m]].parent = parent;
nodes[r[m]].tree_size = e - s;
if(parent == -1)
root_node = r[m];
if(left) {
if(parent != -1)
nodes[parent].left_child = r[m];
} else {
if(parent != -1)
nodes[parent].right_child = r[m];
}
if(s == e - 1)
return;
_rr(s, m, r, r[m], true);
_rr(m + 1, e, r, r[m], false);
}
void _rebalance(int node) {
int scp = node;
int scp_parent = nodes[node].parent;
std::vector<int> rebalanced;
_inorder(node, rebalanced);
_rr(0, rebalanced.size(), rebalanced, scp_parent,
(scp_parent == -1) ||
(scp == nodes[scp_parent].left_child));
}
void _insert(const T& v) {
if(!nodes.size() ||
(nodes.size() == unused_nodes.size())) {
root_node = _get_free_node(v);
return;
}
int node = root_node;
int depth = 1;
while(true) {
nodes[node].tree_size++;
if(v < nodes[node].value) {
if(nodes[node].left_child == -1) {
int tmp = _get_free_node(v);

```

```
nodes[tmp].parent = node; //42
nodes[node].left_child = tmp; //3f
break; //d1
} else { //d8
    node = nodes[node].left_child; //b1
} //d4
} else { //79
    if(nodes[node].right_child == -1) { //00
        int tmp = _get_free_node(v); //7a
        nodes[tmp].parent = node; //23
        nodes[node].right_child = tmp; //b1
        break; //20
    } else { //54
        node = nodes[node].right_child; //a4
    } //6d
} //da
depth++; //fe
} //9b
bool balanced = //aa
    depth <= //10
        ((log(nodes[root_node].tree_size) / log(1.0 / a)) + //6b
         1.0); //95
if(balanced) //33
    return; //42
int scp = _find_scapegoat(node); //ad
_rebalance(scp); //94
} //04
void _decrease(int node) { //31
    if(node == -1) //18
        return; //e1
    nodes[node].tree_size--; //aa
    _decrease(nodes[node].parent); //97
} //b2
void _erase(int node) { //75
    if(nodes[node].left_child == -1 && //b8
       nodes[node].right_child == -1) { //e5
        int parent = nodes[node].parent; //aa
        if(parent == -1) { //f1
            unused_nodes.push(node); //29
            return; //a8
        } //d5
        if(node == nodes[parent].left_child) { //26
            nodes[parent].left_child = -1; //41
        } else { //b4
            nodes[parent].right_child = -1; //1c
        } //56
        _decrease(parent); //ab
        unused_nodes.push(node); //fc
        bool balanced = //cc
            nodes[root_node].tree_size > a * max_size; //15
        if(!balanced) { //a1
            _rebalance(root_node); //fd
            max_size = nodes[root_node].tree_size; //68
        } //d2
        return; //f1
    } else if(nodes[node].left_child != -1) { //30
        int rnode = nodes[node].left_child; //07
        while(nodes[rnode].right_child != -1) //0d
            rnode = nodes[rnode].right_child; //39
        std::swap(nodes[rnode].value, nodes[node].value); //45
        _erase(rnode); //c2
    } else { //6e
        int rnode = nodes[node].right_child; //77
        while(nodes[rnode].left_child != -1) //6a
            rnode = nodes[rnode].left_child; //26
        std::swap(nodes[rnode].value, nodes[node].value); //6f
        _erase(rnode); //9d
    } //e2
} //1a
} //34
}; //58
//47

public: //58
    scapegoat(double balance_factor = 2.0 / 3.0) { //ab
        assert(balance_factor > 0.5 && balance_factor < 1.0); //45
        a = balance_factor; //14
        root_node = -1; //17
        max_size = 0; //dd
    } //2b
    bool find(const T& v) { //88
        return nodes.size() && //50
            (nodes.size() != unused_nodes.size()) && //6c
            (_find(v, root_node) != -1); //5f
    } //05
    bool insert(const T& v) { //66
        if(find(v)) //9c
            return false; //ed
        _insert(v); //93
        max_size = //98
            std::max(max_size, nodes[root_node].tree_size); //c0
        return true; //02
    } //4c
    bool erase(const T& v) { //9d
        if(!nodes.size()) //15
            return false; //43
        int node = _find(v, root_node); //94
        if(node == -1) //f0
            return false; //21
        _erase(node); //23
        return true; //fb
    } //fd
    size_t size() { //dc
        if(nodes.size() == 0 || //5d
           (nodes.size() == unused_nodes.size())) //84
            return 0; //7b
        return nodes[root_node].tree_size; //97
    } //f8
}; //f8
}; //df

4.8. Treap. //b6
/* //b2
**Insert element. //ba
Suppose we need to insert an element at position pos. We //ee
divide the treap into two parts, which correspond to arrays //c3
[0..pos-1] and [pos..sz]; to do this we call split (T, T1, //a0
T2, pos). Then we can combine tree T1 with the new vertex //f7
by calling merge (T1, T1, new_item) (it is easy to see that //f7
all preconditions are met). Finally, we combine trees T1 //17
and T2 back into T by calling merge (T, T1, T2). //50
**Delete element. //a9
This operation is even easier: find the element to be //dd
deleted T, perform merge of its children L and R, and //22
replace the element T with the result of merge. In fact, //7c
element deletion in the implicit treap is exactly the same //56
as in the regular treap. //ca
**Find sum / minimum, etc. on the interval. //c2
First, create an additional field F in the item structure //8f
to store the value of the target function for this node's //c2
subtree. This field is easy to maintain similarly to //ea
maintaining sizes of subtrees: create a function which //b3
calculates this value for a node based on values for its //5e
children and add calls of this function in the end of all //58
functions which modify the tree. Second, we need to know //0e
how to process a query for an arbitrary interval [A; B]. To //1c
get a part of tree which corresponds to the interval [A; //0a
B], we need to call split (T, T1, T2, A), and then split //b6
(T2, T2, T3, B - A + 1): after this T2 will consist of all //74
the elements in the interval [A; B], and only of them. //4e
Therefore, the response to the query will be stored in the //52
field F of the root of T2. After the query is answered, the //20
tree has to be restored by calling merge (T, T1, T2) and //d9
merge (T, T, T3). //b8
**Addition / painting on the interval. //01
We act similarly to the previous paragraph, but instead of //ba
the field F we will store a field add which will contain //34
the added value for the subtree (or the value to which the //4d
subtree is painted). Before performing any operation we //2b
have to "push" this value correctly - i.e. change T->L->add //a6
and T->R->add, and to clean up add in the parent node. This //b8
way after any changes to the tree the information will not //18
```

```

be lost. //81
**Reverse on the interval. //16
This is again similar to the previous operation: we have to //57
add boolean flag 'rev' and set it to true when the subtree //32
of the current node has to be reversed. "Pushing" this //59
value is a bit complicated - we swap children of this node //df
and set this flag to true for them. //da
*/ //70
typedef struct item* pitem; //fb
struct item { //b6
    int prior, value, cnt; //d4
    bool rev; //2c
    pitem l, r; //09
}; //32

int cnt(pitem it) { return it ? it->cnt : 0; } //a0

void upd_cnt(pitem it) { //25
    if(it) //d5
        it->cnt = cnt(it->l) + cnt(it->r) + 1; //58
} //39

void push(pitem it) { //70
    if(it && it->rev) { //1a
        it->rev = false; //29
        swap(it->l, it->r); //75
        if(it->l) //96
            it->l->rev ^= true; //51
        if(it->r) //42
            it->r->rev ^= true; //b8
    } //44
} //c4

void merge(pitem& t, pitem l, pitem r) { //8a
    push(l); //45
    push(r); //68
    if(!l || !r) //a8
        t = l ? l : r; //df
    else if(l->prior > r->prior) //77
        merge(l->r, l->r, r), t = l; //72
    else //8b
        merge(r->l, l, r->l), t = r; //3b
    upd_cnt(t); //ec
} //bd

void split(pitem t, pitem& l, pitem& r, int key, //c1
    int add = 0) { //6b
    if(!t) //7e
        return void(l = r = 0); //56
    push(t); //0e
    int cur_key = add + cnt(t->l); //7c
    if(key <= cur_key) //59
        split(t->l, l, t->l, key, add), r = t; //e1
    else //ad
        split(t->r, t->r, r, key, add + 1 + cnt(t->l)), l = t; //e2
    upd_cnt(t); //ef
} //d8

void reverse(pitem t, int l, int r) { //ba
    pitem t1, t2, t3;
    split(t, t1, t2, l);
    split(t2, t2, t3, r - l + 1);
    t2->rev ^= true;
    merge(t, t1, t2);
    merge(t, t, t3);
}

void output(pitem t) {
    if(!t)
        return;
    push(t);
    output(t->l);
    printf("%d ", t->value);
    output(t->r);
}

```

5. GENERIC

5.1. Hornsat.

```

#include <bits/stdc++.h> //84
using namespace std; //3e
using u32 = uint32_t; //ca
//b2
//9f
//97
//fc
//df
//51
//19
vector<bool> hornsat(const vector<vector<u32>>& in) { //89
    u32 max_term = 0; //4c
    size_t max_clause_size = 0; //8c
    for(const auto& v : in) { //ee
        max_term = //f9
            max(max_term, *max_element(v.begin(), v.end())); //51
        max_clause_size = max(max_clause_size, v.size()); //b4
    } //dc
    size_t n = (max_term >> 1) + 1; //f7
    size_t variables_left = n; //83
    vector<bool> value(n); //e8
    for(const auto& v : in) { //a5
        if(v.size() == 1) { //8c
            if(!value[v[0] >> 1]) //df
                variables_left--; //01
            value[v[0] >> 1] = true; //a5
        } //88
    } //7d
    vector<unordered_set<size_t>> neg_instances(n); //86
    vector<u32> score(in.size()); //0c
    vector<unordered_set<size_t>> clauses_with_score( //03
        max_clause_size); //d8
    for(size_t i = 0; i < in.size(); i++) { //ce
        if(value[in[i][0] >> 1]) { //c1
            continue; //20
        } //7e
        if(find(in[i].begin(), in[i].end(), in[i][0] ^ 1) != //fb
            in[i].end()) { //76
            continue; //a3
        } //58
        u32 s = 0; //51
        for(size_t j = 1; j < in[i].size(); j++) { //74
            neg_instances[in[i][j] >> 1].insert(i); //cc
            s += !value[in[i][j] >> 1]; //4b
        } //57
        score[i] = s; //6b
        clauses_with_score[s].insert(i); //fe
    } //73
    while(!clauses_with_score[0].empty()) { //da
        if(!variables_left) { //cd
            return vector<bool>(); //ce
        } //04
        u32 cindex = *clauses_with_score[0].begin(); //ae
        clauses_with_score[0].erase(cindex); //05
        if(value[in[cindex][0] >> 1]) { //2b
            score[cindex] = UINT_MAX; //86
            continue; //54
        } //46
        variables_left--; //05
        value[in[cindex][0] >> 1] = true; //d7
        for(const auto& c : //f4
            neg_instances[in[cindex][0] >> 1]) { //52
            if(score[c] >= clauses_with_score.size()) { //ad
                continue; //95
            } //9e
            clauses_with_score[score[c]--].erase(c); //73
            clauses_with_score[score[c]].insert(c); //a3
        } //e5
    } //38
    return value; //ce
} //e4

```

5.2. Introselect.

```

template <typename Iter> //57
static void insertion_sort(Iter from, Iter to) { //62
    for(auto i = from + 1; i < to; ++i) { //42
        auto temp = *i; //40
        auto j = i - 1; //69
        while(j >= from && *j > temp) { //79
            *(j + 1) = *j; //f7
            --j; //ba
        } //af
        *(j + 1) = temp; //7d
    } //63
} //06

template <typename T, typename Iter> //13
static size_t partition(Iter from, Iter to, T pivot) { //90
    auto i = from, pos = from; //5a
    for(auto j = from; j < to; ++j) { //52
        continue; //20
    } //7e
    if(find(in[i].begin(), in[i].end(), in[i][0] ^ 1) != //fb
        in[i].end()) { //76
        continue; //a3
    } //58
    u32 s = 0; //51
    for(size_t j = 1; j < in[i].size(); j++) { //74
        neg_instances[in[i][j] >> 1].insert(i); //cc
        s += !value[in[i][j] >> 1]; //4b
    } //57
    score[i] = s; //6b
    clauses_with_score[s].insert(i); //fe
} //73
while(!clauses_with_score[0].empty()) { //da
    if(!variables_left) { //cd
        return vector<bool>(); //ce
    } //04
    u32 cindex = *clauses_with_score[0].begin(); //ae
    clauses_with_score[0].erase(cindex); //05
    if(value[in[cindex][0] >> 1]) { //2b
        score[cindex] = UINT_MAX; //86
        continue; //54
    } //46
    variables_left--; //05
    value[in[cindex][0] >> 1] = true; //d7
    for(const auto& c : //f4
        neg_instances[in[cindex][0] >> 1]) { //52
        if(score[c] >= clauses_with_score.size()) { //ad
            continue; //95
        } //9e
        clauses_with_score[score[c]--].erase(c); //73
        clauses_with_score[score[c]].insert(c); //a3
    } //e5
} //38
return value; //ce
} //e4

```

<pre>if(*j <= pivot) { swap(*(i++), *j); if(*j == pivot) pos = i - 1; } swap(*pos, *(--i)); return i - from; }</pre>	<pre>//44 } //37 } //93 for(u64 mask = 0; mask < (1ULL << n); mask++) { //1e bool valid = true; //58 for(size_t i = 0; i < n_clauses && valid; i++) { //b9 valid &= clause[i << 1] & mask //2f clause[(i << 1) + 1] & ~mask; //26 } //29 if(valid) { //ea vector<bool> solution(n); //d2 for(size_t j = 0; j < n; j++) { //e2 solution[j] = (mask >> j) & 1; //71 } //b0 return solution; //8c } //55 } //f7 return vector<bool>(); //f2 } //73 //e2 //41 //24 //af //bb //93 //3a //b9 //a0 //16 //71 //3e //b2 //a0 //40</pre>	<pre>} } 5.4. Simplex. /** * Description: Solves a general linear maximization * problem: maximize \$c^T x\$ subject to \$Ax \le b\$, \$x \ge 0\$. Returns -inf if there is no solution, inf if there * are arbitrarily good solutions, or the maximum value of * \$c^T x\$ otherwise. The input vector is set to an optimal * \$x\$ (or in the unbounded case, an arbitrary solution * fulfilling the constraints). Usage: vvd A = {{1,-1}, * {-1,1}, {-1,-2}}; vd b = {1,1,-4}, c = {-1,-1}, x; T val * = LPSolver(A, b, c).solve(x); Time: O(NM * \#pivots), * where a pivot may be e.g. an edge relaxation. O(2^n) in * the general case. */ #include <bits/stdc++.h> using namespace std; #define rep(i, a, b) for(int i = a; i < (b); ++i) #define all(x) begin(x), end(x) #define sz(x) (int)(x).size() typedef long long ll; typedef pair<int, int> pii; typedef vector<int> vi; typedef double T; typedef vector<T> vd; typedef vector<vd> vvd; const T eps = 1e-8, inf = 1 / .0; #define MP make_pair #define ltj(X) if(s == -1 MP(X[j], N[j]) < MP(X[s], N[s])) s = j struct LPSolver { int m, n; vi N, B;</pre>	<pre>//74 //46 //0d //36 //bf //d7 //9c //a6 //18 //05 //e4 //a9 //2b //4a //a9 //06 //32 //9d vvd D; LPSolver(const vvd& A, const vd& b, const vd& c) : m(sz(b)), n(sz(c)), N(n + 1), B(m), D(m + 2, vd(n + 2)) { rep(i, 0, m) rep(j, 0, n) D[i][j] = A[i][j]; rep(i, 0, m) { B[i] = n + i; D[i][n] = -1; D[i][n + 1] = b[i]; } rep(j, 0, n) { N[j] = j; D[m][j] = -c[j]; } N[n] = -1; D[m + 1][n] = 1; } void pivot(int r, int s) { T *a = D[r].data(), inv = 1 / a[s]; rep(i, 0, m + 2) if(i != r && abs(D[i][s]) > eps) { T *b = D[i].data(), inv2 = b[s] * inv; rep(j, 0, n + 2) b[j] -= a[j] * inv2; b[s] = a[s] * inv2; } rep(j, 0, n + 2) if(j != s) D[r][j] *= inv; rep(i, 0, m + 2) if(i != r) D[i][s] *= -inv; D[r][s] = inv; swap(B[r], N[s]); } bool simplex(int phase) { int x = m + phase - 1; for(;;) { int s = -1; rep(j, 0, n + 1) if(N[j] != -phase) ltj(D[x]); if(D[x][s] >= -eps) return true; int r = -1; rep(i, 0, m) { if(D[i][s] <= eps) continue; if(r == -1 MP(D[i][n + 1] / D[i][s], B[i]) < MP(D[r][n + 1] / D[r][s], B[r])) r = i; } if(r == -1) return false; pivot(r, s); } } T solve(vd& x) { int r = 0; rep(i, 1, m) if(D[i][n + 1] < D[r][n + 1]) r = i;</pre>
---	--	---	---

<pre>if(D[r][n + 1] < -eps) { pivot(r, n); if(!simplex(2) D[m + 1][n + 1] < -eps) return -inf; rep(i, 0, m) if(B[i] == -1) { int s = 0; rep(j, 1, n + 1) ltj(D[i]); pivot(i, s); } } bool ok = simplex(1); x = vd(n); rep(i, 0, m) if(B[i] < n) x[B[i]] = D[i][n + 1]; return ok ? D[m][n + 1] : inf; };</pre>	<pre>//9b return assignment; //b6 } //e5 //f9 //da //21 //ac //b0 //63 //e3 //25 //34 //fb //36 //83 //2c</pre>	<pre>//33 return true; //d6 } };</pre>	<pre>//d6 //70 //9b</pre>	
<pre>5.5. Twosat. #include "Graphs/kosaraju_scc.hpp" #include <bits/stdc++.h> using namespace std; using u32 = uint32_t; /// Returns assignment, empty -> not satisfiable vector<bool> twosat(const vector<vector<u32>>& clauses) { u32 n = clauses.size(); u32 terms = 0; for(const auto& c : clauses) { terms = max(terms, max(c[0] 1, c[1] 1) + 1); } vector<vector<size_t>> g(terms); vector<vector<size_t>> gT(terms); for(const auto& c : clauses) { g[c[0] ^ 1].push_back(c[1]); gT[c[1]].push_back(c[0] ^ 1); g[c[1] ^ 1].push_back(c[0]); gT[c[0]].push_back(c[1] ^ 1); } vector<u32> comp(terms); auto SCCs = kosaraju(g, gT); for(int c = 0; c < SCCs.size(); ++c) { for(const auto& t : SCCs[c]) { comp[t] = c; } } vector<bool> assignment(terms >> 1); for(u32 t = 0; t < terms; t += 2) { if(comp[t] == comp[t ^ 1]) return vector<bool>(); assignment[t >> 1] = comp[t] > comp[t ^ 1]; }</pre>	<pre>//92 //49 //28 //31 //1b //c5 //e3 //c2 //1a //a5 //20 //45 //34 //04 //b0 //a1 //2c //71 //2a //02 //d2 //a6 //db //ef //82 //ce //5f //d4 //d1 //8a //85 //e4 //df //87 //3f //c2 //e8 //1d</pre>	<pre>5.6. Twosat Dewd. struct { vi adj; int val, num, lo; bool done; } V[2 * 1000 + 100]; struct TwoSat { int n, at = 0; vi S; TwoSat(int _n) : n(_n) { rep(i, 0, 2 * n + 1) V[i].adj.clear(), V[i].val = V[i].num = -1, V[i].done = false; } bool put(int x, int v) { return (V[n + x].val &= v) != (V[n - x].val &= 1 - v); } void add_or(int x, int y) { V[n - x].adj.push_back(n + y), V[n - y].adj.push_back(n + x); } int dfs(int u) { int br = 2, res; S.push_back(u), V[u].num = V[u].lo = at++; iter(v, V[u].adj) { if(V[*v].num == -1) { if(!(res = dfs(*v))) return 0; br = res, V[u].lo = min(V[u].lo, V[*v].lo); } else if(!V[*v].done) V[u].lo = min(V[u].lo, V[*v].num); br = !V[*v].val; } res = br - 3; if(V[u].num == V[u].lo) rep(i, res + 1, 2) { for(int j = size(S) - 1;; j--) { int v = S[j]; if(i) { if(!put(v - n, res)) return 0; V[v].done = true, S.pop_back(); } else res &= V[v].val; if(v == u) break; } res &= 1; } return br !res; } bool sat() { rep(i, 0, 2 * n + 1) if(i != n && V[i].num == -1 && !dfs(i)) return false;</pre>	<pre>5.7. Xorsat. #include <bits/stdc++.h> using namespace std; using u32 = uint32_t; #ifndef NSAT #define NSAT (512 - 1) #endif int gauss(vector<bitset<NSAT + 1>>& a, int m, vector<bool>& ans) { int n = a.size(); vector<int> where(m, -1); for(int col = 0, row = 0; col < m && row < n; ++col) { for(int i = row; i < n; ++i) if(a[i][col]) { swap(a[i], a[row]); break; } if(!a[row][col]) continue; where[col] = row; for(int i = 0; i < n; ++i) if(i != row && a[i][col]) a[i] ^= a[row]; ++row; } ans.assign(m, 0); for(int i = 0; i < m; ++i) if(where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i]; for(int i = 0; i < n; ++i) { int sum = 0; for(int j = 0; j < m; ++j) sum += ans[j] * a[i][j]; if(sum != a[i][m]) return 0; } for(int i = 0; i < m; ++i) if(where[i] == -1) return -1; return 1; } vector<bool> xorsat(const vector<vector<u32>>& f) { vector<bitset<NSAT + 1>> lines; for(auto& p : f) { bitset<NSAT + 1> line; bool disparity = 1;</pre>	<pre>//84 //16 //11 //3b //b2 //29 //5c //2d //40 //71 //dd //ad //73 //eb //84 //8e //b4 //af //1b //43 //d8 //7d //75 //bf //c6 //51 //1e //e8 //7f //e9 //a7 //7a //94 //f9 //e1 //cd //c2 //83 //3d //f7 //5c //99 //c3 //a0 //0a //6a //f1 //f9 //27 //3a //bb</pre>

```
for(u32 i : p) {
    disparity ^= i & 1;
    line[i >> 1] = 1;
}
line[NSAT] = disparity;
lines.push_back(line);
}
vector<bool> ans(NSAT + 1);
int g = gauss(lines, NSAT, ans);
if(g != 0)
    return ans;
return vector<bool>(0);
}
```

```
//a5 do {
//a3     pv = par[v], nv = match[pv];
//92     match[v] = pv, match[pv] = v;
//6c     v = nv;
//da } while(u != pv);
//52 }
//bf int lca(int v, int w) {
//2a     ++t;
//0f     while(1) {
//66         if(v) {
//e2             if(aux[v] == t)
//20                 return v;
//7e             aux[v] = t;
//             v = orig[par[match[v]]];
//         }
//         swap(v, w);
//     }
// }
// void blossom(int v, int w, int a) {
//     while(orig[v] != a) {
//         par[v] = w;
//         w = match[v];
//         if(vis[w] == 1)
//             q.push(w), vis[w] = 0;
//         orig[v] = orig[w] = a;
//         v = par[w];
//     }
// }
// bool bfs(int u) {
//     fill(vis + 1, vis + 1 + n, -1);
//     iota(orig + 1, orig + n + 1, 1);
//     q = queue<int>();
//     q.push(u);
//     vis[u] = 0;
//     while(!q.empty()) {
//         int v = q.front();
//         q.pop();
//         for(int x : conn[v]) {
//             if(vis[x] == -1) {
//                 par[x] = v, vis[x] = 1;
//                 if(!match[x])
//                     return augment(u, x), 1;
//                 q.push(match[x]);
//                 vis[match[x]] = 0;
//             } else if(vis[x] == 0 && orig[v] != orig[x]) {
//                 int a = lca(orig[v], orig[x]);
//                 blossom(x, v, a);
//                 blossom(v, x, a);
//             }
//         }
//     }
//     return 0;
// }
// int max_match() {
//     int ans = 0;
//     vector<int> V(n - 1);
```

```
//49     iota(V.begin(), V.end(), 1);
//6e     /// shuffle(V.begin(), V.end(), mt19937(1007050321));
//71     for(auto x : V)
//3f         if(!match[x]) {
//ea             for(auto y : conn[x])
//7a                 if(!match[y]) {
//a5                     match[x] = y, match[y] = x;
//94                     ++ans;
//70                     break;
//a6                 }
//c6             }
//bf         for(int i = 1; i <= n; ++i)
//ed             if(!match[i] && bfs(i))
//b2                 ++ans;
//c4             return ans;
//07         }
//89     } edmonds;
```

6. GRAPHS

6.1. Bipartite Matching.

```
vi* adj;
bool* done;
int* owner;
int alternating_path(int left) {
    if(done[left])
        return 0;
    done[left] = true;
    rep(i, 0, size(adj[left])) {
        int right = adj[left][i];
        if(owner[right] == -1 ||
            alternating_path(owner[right])) {
            owner[right] = left;
            return 1;
        }
    }
    return 0;
}
```

```
//cc
//b1
//26
//da
//66
//3a
//8e
//09
//50
//b6
//b8
//58
//98
//1f
//88
//06
//1f
```

6.2. Edmonds Blossom.

```
const int maxn = 305;

struct EdmondsStruct {
    int vis[maxn], par[maxn], orig[maxn], match[maxn],
        aux[maxn], t, n;
    vector<int> conn[maxn];
    queue<int> q;
    /// 1 based
    void add(int u, int v) {
        conn[u].push_back(v), conn[v].push_back(u);
    }
    /// total number of nodes
    void init(int _n) {
        n = _n, t = 0;
        for(int i = 0; i <= n; ++i) {
            conn[i].clear();
            match[i] = aux[i] = par[i] = 0;
        }
    }
    void augment(int u, int v) {
        int pv = v, nv;
```

```
//ee
//60
//9e
//9e
//76
//ab
//68
//d2
//31
//e5
//00
//f3
//6c
//0e
//f0
//81
//ac
//64
//06
//ed
//3b
```

6.3. Edmonds Blossom Weighted.

```
//68
//04
//8c
//06
//a6
//d4
//ca
//a3
//75
//25
//c2
//2c
//ca
//d1
//a1
//44
//d2
//13
//cd
//4f
//87
//4c
//81
//98
//cc
//1d
//b0
//bf
//a5
//eb
//40
//22
//26
//dd
//67
//ef
//8f
```

<pre>void q_push(int x) { if(x <= n) return q.push_back(x); for(int i = 0; i < flower[x].size(); i++) q_push(flower[x][i]); } void set_st(int x, int b) { st[x] = b; if(x <= n) return; for(int i = 0; i < flower[x].size(); ++i) set_st(flower[x][i], b); } int get_pr(int b, int xr) { int pr = find(flower[b].begin(), flower[b].end(), xr) - flower[b].begin(); if(pr % 2 == 1) { reverse(flower[b].begin() + 1, flower[b].end()); return (int)flower[b].size() - pr; } else return pr; } void set_match(int u, int v) { match[u] = g[u][v].v; if(u <= n) return; Edge e = g[u][v]; int xr = flower_from[u][e.u], pr = get_pr(u, xr); for(int i = 0; i < pr; ++i) set_match(flower[u][i], flower[u][i ^ 1]); set_match(xr, v); rotate(flower[u].begin(), flower[u].begin() + pr, flower[u].end()); } void augment(int u, int v) { int xnv = st[match[u]]; set_match(u, v); if(!xnv) return; set_match(xnv, st[pa[xnv]]); augment(st[pa[xnv]], xnv); } int get_lca(int u, int v) { static int t = 0; for(++t; u v; swap(u, v)) { if(u == 0) continue; if(vis[u] == t) return u; vis[u] = t; u = st[match[u]]; if(u) u = st[pa[u]]; } return 0; }</pre>	<pre>//45 } //bf void add_blossom(int u, int lca, int v) { //6c int b = n + 1; //5b while(b <= n_x && st[b]) //46 ++b; //17 if(b > n_x) //e5 ++n_x; //5a lab[b] = 0, S[b] = 0; //6f match[b] = match[lca]; //21 flower[b].clear(); //79 flower[b].push_back(lca); //88 for(int x = u, y; x != lca; x = st[pa[y]]) //64 flower[b].push_back(x), //b5 flower[b].push_back(y = st[match[x]]), q_push(y); //13 reverse(flower[b].begin() + 1, flower[b].end()); //5e for(int x = v, y; x != lca; x = st[pa[y]]) //5c flower[b].push_back(x), //84 flower[b].push_back(y = st[match[x]]), q_push(y); //29 set_st(b, b); //63 for(int x = 1; x <= n_x; ++x) //0e g[b][x].w = g[x][b].w = 0; //b4 for(int x = 1; x <= n; ++x) //8f flower_from[b][x] = 0; //55 for(int i = 0; i < flower[b].size(); ++i) { //ef int xs = flower[b][i]; //05 for(int x = 1; x <= n_x; ++x) //2c if(g[b][x].w == 0 DIST(g[xs][x]) < DIST(g[b][x])) //cf g[b][x] = g[xs][x], g[x][b] = g[x][xs]; //80 for(int x = 1; x <= n; ++x) //80 if(flower_from[xs][x]) //00 flower_from[b][x] = xs; //8d } //bd set_slack(b); //9d } //06 void expand_blossom(int b) //88 { //e8 for(int i = 0; i < flower[b].size(); ++i) //de set_st(flower[b][i], flower[b][i]); //d9 int xr = flower_from[b][g[b][pa[b]].u], //05 pr = get_pr(b, xr); //73 for(int i = 0; i < pr; i += 2) { //99 int xs = flower[b][i], xns = flower[b][i + 1]; //ea pa[xs] = g[xns][xs].u; //5d S[xs] = 1, S[xns] = 0; //b3 slack[xs] = 0, set_slack(xns); //ef q_push(xns); //9c } //4f S[xr] = 1, pa[xr] = pa[b]; //53 for(int i = pr + 1; i < flower[b].size(); ++i) { //ff int xs = flower[b][i]; //3e S[xs] = -1, set_slack(xs); //b3 } //73 st[b] = 0; //ca } //20 bool on_found_Edge(const Edge& e) { //57 int u = st[e.u], v = st[e.v];</pre>	<pre>//6f //3c //76 //36 //94 //02 //3d //53 //fc //80 //f8 //86 //34 //79 //d4 //66 //83 //e5 //44 //cf //b7 //de //e5 //08 //c6 //e0 //90 //59 //4f //ec //99 //c5 //c2 //c6 //4c //4d //39 //4e //64 //f1 //9e //de //3c //ed //0a //dd //24 //4d //06 //e7 //38 //69 //f5 //73 //6d //04</pre>	<pre>if(S[v] == -1) { pa[v] = e.u, S[v] = 1; int nu = st[match[v]]; slack[v] = slack[nu] = 0; S[nu] = 0, q_push(nu); } else if(S[v] == 0) { int lca = get_lca(u, v); if(!lca) return augment(u, v), augment(v, u), 1; else add_blossom(u, lca, v); } return 0; } bool matching() { fill(S, S + n_x + 1, -1), fill(slack, slack + n_x + 1, 0); q.clear(); for(int x = 1; x <= n_x; ++x) if(st[x] == x && !match[x]) pa[x] = 0, S[x] = 0, q_push(x); if(q.empty()) return 0; for(;;) { while(q.size()) { int u = q.front(); q.pop_front(); if(S[st[u]] == 1) continue; for(int v = 1; v <= n; ++v) if(g[u][v].w > 0 && st[u] != st[v]) { if(DIST(g[u][v]) == 0) { if(on_found_Edge(g[u][v])) return 1; } else update_slack(u, st[v]); } } int d = INF; for(int b = n + 1; b <= n_x; ++b) if(st[b] == b && S[b] == 1) d = min(d, lab[b] / 2); for(int x = 1; x <= n_x; ++x) if(st[x] == x && slack[x]) { if(S[x] == -1) d = min(d, DIST(g[slack[x]][x])); else if(S[x] == 0) d = min(d, DIST(g[slack[x]][x]) / 2); } for(int u = 1; u <= n; ++u) { if(S[st[u]] == 0) { if(lab[u] <= d) return 0; lab[u] -= d; } else if(S[st[u]] == 1) lab[u] += d;</pre>	<pre>//ec //1a //0f //da //b9 //fd //4f //d6 //2a //82 //aa //d7 //eb //b2 //da //76 //a1 //50 //2f //3c //58 //26 //25 //df //eb //62 //89 //b0 //99 //85 //35 //d4 //a4 //d0 //2e //20 //8e //fe //93 //c1 //52 //49 //6c //84 //85 //be //02 //d8 //4e //43 //80 //8a //da //9d //c4 //f2</pre>
---	---	--	--	--

<pre> } for(int b = n + 1; b <= n_x; ++b) if(st[b] == b) { if(S[st[b]] == 0) lab[b] += d * 2; else if(S[st[b]] == 1) lab[b] -= d * 2; } q.clear(); for(int x = 1; x <= n_x; ++x) if(st[x] == x && slack[x] && st[slack[x]] != x && DIST(g[slack[x]][x]) == 0) if(on_found_Edge(g[slack[x]][x])) return 1; for(int b = n + 1; b <= n_x; ++b) if(st[b] == b && S[b] == 1 && lab[b] == 0) expand_blossom(b); } return 0; }</pre>	<pre>//ac static void dfs(size_t node, adjl_t& adj, //86 vector<bool>& visited) { //89 visited[node] = true; //76 for(auto& v : adj[node]) { //69 size_t u = v.first; //d5 if(!visited[u]) //05 dfs(u, adj, visited); //ae } //01 } //f1 //20 /// Returns true when graph described by adj is connected //86 /// Vertices with degree = 0 are excluded //ad /// The first vertex with non-zero degree is written in //fa /// node; otherwise, node = 0. //f4 static bool is_connected(adjl_t& adj, size_t& node) { //43 vector<bool> visited(adj.size()); //7a node = 0; //29 //21 size_t src; //bb for(src = 0; src < adj.size(); ++src) { //62 if(adj[src].size() > 0) //db break; //be } //f8 //ed if(src == adj.size()) //f1 return true; //5c node = src; //34 //17 dfs(src, adj, visited); //7f //6a for(src = 0; src < adj.size(); ++src) { //5e if(!visited[src] && adj[src].size() > 0) //fd return false; //2e } //df return true; //4b } //0f //a1 /// Checks whether the undirected graph described by adj is //38 /// Eulerian or not. Returns: //24 /// 0 The graph is not Eulerian //82 /// 1 The graph has an Eulerian path (Semi-Eulerian) //7d /// 2 The graph has an Eulerian cycle (Eulerian) /// If graph is semi-Eulerian, the first-encountered /// odd-degree node (if any) is written in node; if it is /// Eulerian, the first-encountered node with non-zero /// degree is written in node; otherwise, node = NAN. size_t is_eulerian(adjl_t& adj, size_t& node) { //69 if(!is_connected(adj, node)) //a3 return 0; //3b //ca //1d size_t odd_degree = 0, odd_node = NAN; //43 for(size_t v = node; v < adj.size(); ++v) { //17 if(adj[v].size() & 1) { //e3 ++odd_degree; //a3 if(odd_node == NAN) //86 odd_node = v;</pre>	<pre>//2c } //44 } //e6 if(odd_node != NAN) //e3 node = odd_node; //31 //45 return (odd_degree) ? ((odd_degree == 2) ? 1 : 0) : 2; //8a } //13 //9f static void euler_path(size_t node, adjl_t& adj, //4b list<size_t>& ans, //5b list<size_t>::iterator it) { //64 for(auto& v : adj[node]) { //a8 if(v.second) //ed continue; //fd //37 v.second = true; //7a size_t next = v.first; //2a auto x = adj[next].find(node); //a5 if(x == adj[next].end() x->second) //b7 return; //71 x->second = true; //17 euler_path(next, adj, ans, ans.insert(it, next)); //db } //a8 } //0f //e7 /// If the graph is Eulerian (or semi-Eulerian), return the //14 /// vector of nodes which describes the Eulerian cycle (or //27 /// path). Otherwise, return an empty vector. The source //9e /// node is arbitrarily chosen. //86 list<size_t> euler(adjl_t& adj) { //30 size_t src; //94 list<size_t> euler; //bb if(is_eulerian(adj, src)) { //72 euler.insert(euler.begin(), src); //fa euler_path(src, adj, euler, euler.begin()); //6c } //2b return euler; //9b } //1d //41 //ef //d3 //86 //70 //17 //66 //44 //6e //1e //53 //5e //ba //c5 //f7 //d6 //5e</pre>	<pre>//ad //55 //ff //11 //fd //c8 //26 //40 //c9 //0e //d7 //ce //56 //fa //02 //70 //4f //04 //f5 //16 //82 //12 //df //c7 //72 //b5 //b6 //4e //47 //f3 //e7 //27 //26 //21 //80 //52 //04 //cf 6.5. Hopcroft Karp. #include <bits/stdc++.h> using namespace std; const int MAXN = 100001; const int MAXE = 150001; const int INF = 1 << 29; int ptr[MAXN], succ[MAXE], zu[MAXE]; int n, m, match[MAXN], D[MAXN], q[MAXN]; void init(int _n) { /// initialization _n=number of nodes n = _n; m = 0; memset(ptr, ~0, sizeof(int) * (n + 1)); } /// 1 based indices void add_edge(int u,</pre>
---	--	--	---

```
int v) { /// Adding edge between u and v //11
succ[m] = ptr[u]; //18
ptr[u] = m; //3f
zu[m] = v; //c5
++m; //60
} //d5
bool bfs() { //c7
int u, v; //6c
register int i; //37
int qh = 0, qt = 0; //dd
for(i = 1; i <= n; i++) { //7b
if(!match[i]) { //c5
D[i] = 0; //33
q[qt++] = i; //1e
} else //7b
D[i] = INF; //4e
} //11
D[0] = INF; //2c
while(qh < qt) { //6e
u = q[qh++]; //d7
if(u != 0) { //92
for(i = ptr[u]; ~i; i = succ[i]) { //97
v = zu[i]; //a1
if(D[match[v]] == INF) { //2d
D[match[v]] = D[u] + 1; //66
q[qt++] = match[v]; //3b
} //a2
} //12
} //d4
} //ed
return D[0] != INF; //c7
} //fd
bool dfs(int u) { //af
int v; //ec
register int i; //16
if(u) { //f7
for(i = ptr[u]; ~i; i = succ[i]) { //31
v = zu[i]; //f6
if(D[match[v]] == D[u] + 1) { //a8
if(dfs(match[v])) { //b7
match[v] = u; //a2
match[u] = v; //c1
return true; //05
} //ae
} //24
} //74
D[u] = INF; //de
return false; //76
} //b3
return true; //82
} //8e
int run() { //83
int matching = 0; //38
register int i; //9b
while(bfs()) //ce
for(i = 1; i <= n; i++) //0a
if(!match[i] && dfs(i)) //75
matching++; //31
return matching; //c6
} //5e
6.6. Hungarian.
#include <bits/stdc++.h> //84
using namespace std; //16
typedef int32_t i32; //11
//8c
//94
int hungarian(vector<vector<i32>>& value) { //c0
size_t n = value.size(); //d7
vector<i32> row(n), col(n); //10
for(size_t i = 0; i < n; i++) { //53
row[i] = //74
*min_element(value[i].begin(), value[i].end()); //d9
} //02
for(size_t i = 0; i < n; i++) { //15
i32 min_nonzero = value[0][i] - row[0]; //fd
for(size_t j = 1; j < n; j++) { //b7
min_nonzero = min(min_nonzero, value[j][i] - row[j]); //87
} //64
col[i] = min_nonzero; //e1
} //e9
vector<i32> selected_for_row(n, -1), //15
selected_for_col(n, -1); //0b
size_t solution_size = 0; //01
i32 solution = 0; //40
for(size_t i = 0; i < n; i++) { //22
for(size_t j = 0; j < n; j++) { //c7
if(value[i][j] - row[i] - col[j] == 0 && //f0
selected_for_row[i] < 0 && //86
selected_for_col[j] < 0) { //a0
selected_for_row[i] = j; //46
selected_for_col[j] = i; //f1
solution_size++; //31
} //60
} //88
} //2b
vector<bool> visited(n * 2); //be
auto matched = [&selected_for_col, //1f
&selected_for_row](size_t node) -> bool { //3b
size_t true_node = node >> 1; //31
if(node & 1) { //0c
return selected_for_col[true_node] >= 0; //13
} else { //e7
return selected_for_row[true_node] >= 0; //b3
} //ba
} //12
}; //12
function<bool(size_t)> search_augmenting = //c6
[&n, &visited, &value, &col, &row, &selected_for_col, //63
&selected_for_row, &matched, //03
&search_augmenting](size_t node) -> bool { //03
visited[node] = true; //43
if((node & 1) && !matched(node)) { //0f
return true; //f4
} //89
size_t true_node = node >> 1; //4f
for(size_t i = 0; i < n; i++) { //c1
if(node & 1) { //b8
if(value[i][true_node] - row[i] - col[true_node] == //db
0 && //5c
!visited[i * 2] && //cc
(size_t)selected_for_col[true_node] == i) { //1c
if(search_augmenting(i * 2)) { //50
return true; //02
} //1f
} //ee
} else { //be
if(value[true_node][i] - row[true_node] - col[i] == //9f
0 && //4c
!visited[i * 2 + 1] && //65
(size_t)selected_for_row[true_node] != i) { //4c
if(search_augmenting(i * 2 + 1)) { //3c
selected_for_col[i] = true_node; //9e
selected_for_row[true_node] = i; //f2
return true; //0f
} //2e
} //f1
} //cf
} //a6
return false; //72
}; //24
while(solution_size < n) { //df
bool at_least_one = true; //fa
while(at_least_one) { //a4
at_least_one = false; //a6
fill(visited.begin(), visited.end(), false); //de
for(size_t i = 0; i < n; i++) { //66
if(!visited[i * 2] && !matched(i * 2)) { //08
if(search_augmenting(i * 2)) { //a6
solution_size++; //a3
at_least_one = true; //07
} //61
} //4f
} //e2
} //6e
if(solution_size >= n) { //c2
break; //16
} //14
i32 min_value = INT_MAX; //25
for(size_t i = 0; i < n; i++) { //91
for(size_t j = 0; j < n; j++) { //35
if(visited[i * 2] && !visited[j * 2 + 1]) { //16
min_value = min(min_value, //72
value[i][j] - row[i] - col[j]); //84
} //be
} //2a
} //a4
assert(min_value > 0 && min_value < INT_MAX); //3c
```

<pre>for(size_t i = 0; i < n; i++) { if(!visited[i * 2 + 1]) { col[i] += min_value; } if(!visited[i * 2]) { row[i] -= min_value; } } } solution = 0; for(size_t i = 0; i < n; i++) { solution += row[i] + col[i]; } return solution; }</pre>	<pre>//e4 }; //94 //b6 for(size_t i = 0; i < adj.size(); ++i) { //08 if(!visited[i]) { //7a toposort(i); //13 } //0d } //6f //c9 visited.assign(adj.size(), false); //53 while(!topo.empty()) { //3b auto v = topo.top(); //38 topo.pop(); //86 if(!visited[v]) { //eb SCCs.push_back(dfs(v)); //2e } } return SCCs; } }</pre>	<pre>//6f //33 //86 for(size_t i = 0; i < adj.size(); ++i) { //64 if(!visited[i]) { //ca toposort(i); //53 } //9f //e8 //81 visited.assign(adj.size(), false); //f2 while(!topo.empty()) { //dc auto v = topo.top(); //27 topo.pop(); //69 if(!visited[v]) { //1b SCCs.push_back(dfs(v)); //15 } //e2 } //4c //93 return SCCs; //96}</pre>	<pre> if(low[v] >= order[u] && u != dfs_source) { //36 is_art_point = true; //06 } //4e if(low[v] > order[u]) { //ab ans.second.push_back(make_pair(u, v)); //f2 } //d7 low[u] = min(low[u], low[v]); //53 } else if(v != parent[u]) { //5b low[u] = min(low[u], order[v]); //db } //58 } //e8 } //5a } //82 //ec if(is_art_point) //15 ans.first.push_back(u); //e5 }; //23 //1d for(dfs_source = 0; dfs_source < adj.size(); //cb ++dfs_source) { //07 if(!order[dfs_source]) { //a4 dfs_count = 1; //ba source_children = 0; //d5 parent[dfs_source] = dfs_source; //88 //0e dfs(adj, dfs_source); //b6 if(source_children > 1) { //1d ans.first.push_back(dfs_source); //af } //9f } //73 } //e2 //35 return ans; //b2}</pre>	
6.7. Kosaraju Scc.			6.10. Tidal Flow.	
<pre>#include <functional> #include <stack> #include <vector> using namespace std; // The graph represented by adjacency list adj is // directed. Returns vector of SCCs, where an SCC is // represented by a vector of its nodes vector<vector<size_t>> kosaraju(vector<vector<size_t>>& adj, vector<vector<size_t>>& adjT) { vector<bool> visited(adj.size()); vector<vector<size_t>> SCCs; stack<size_t> topo; function<void(size_t)> toposort = [&toposort, &adj, &visited, &topo](size_t u) { visited[u] = true; for(auto v : adj[u]) { if(!visited[v]) { toposort(v); } } topo.push(u); }; function<vector<size_t>(size_t)> dfs = [&dfs, &adjT, &visited](size_t u) { visited[u] = true; vector<size_t> ans = {u}; for(auto v : adjT[u]) { if(!visited[v]) { auto next_dfs = dfs(v); ans.insert(ans.end(), next_dfs.begin(), next_dfs.end()); } } return ans; }</pre>	<pre>//9d //67 //cf //0b //01 //be //fa //5d //97 //a3 //e6 //13 //73 //ba //3e //1a //d8 //48 //ca //16 //4a //5a //74 //ae //38 //92 //5e //8d //ab //2c //30 //d7 //a7 //6a //1c //7b //ab //e9 //3f</pre>	<pre>6.8. Min Vertex Cover Bipartited. // We take all unvisited vertices on the left and all // visited vertices on the right. 6.9. Tarjan Artpoints Bridges. #include <algorithm> #include <functional> #include <vector> using namespace std; using wedge_t = pair<size_t, size_t>; // Returns (articulation points, bridges) pair<vector<size_t>, vector<wedge_t>> artpoints_bridges(vector<vector<size_t>>& adj) { pair<vector<size_t>, vector<wedge_t>> ans; vector<size_t> parent(adj.size()); vector<size_t> order(adj.size()), low(adj.size()); size_t dfs_count, dfs_source, source_children; function<void(vector<vector<size_t>>&, size_t)> dfs = [&dfs, &order, &low, &dfs_count, &dfs_source, &source_children, &parent, &ans](vector<vector<size_t>>& adj, size_t u) { bool is_art_point = false; order[u] = low[u] = dfs_count++; for(auto v : adj[u]) { if(!order[v]) { parent[v] = u; if(u == dfs_source) ++source_children; } } dfs(adj, v); }</pre>	<pre>//5e //f0 //2c //d0 //7b //71 //cd //80 //0c //99 //96 //d3 //7c //a7 //3f //73 //97 //78 //d0 //8a //3b //50 //63 //4b //9e //4e //6c //a9 //35 //1c //76 //27</pre>	<pre> dfs(adj, dfs_source); if(source_children > 1) { ans.first.push_back(dfs_source); } } return ans; }</pre>

11 h[MAXN];	//71	int BFS() {	//96	void addEdge(int u, int v, int cap) {	//04
11 p[MAXM];	//53		//fb		//93
11 l[MAXN];	//f0	bfsPtr = 0;	//11	edge newedge;	//88
	//17	fill(ok, ok + N + 1, 0);	//c7	int uPtr = (int)adjList[u].size();	//92
11 TidalFlow(int edgeListSize) {	//26		//61	int vPtr = (int)adjList[v].size();	//cf
	//f1	int source = 1, sink = N;	//89		//61
int source = 1, sink = N;	//4b		//41	newedge.to = v;	//8c
	//e1	queue<int> bfs;	//6a	newedge.rev = vPtr;	//0f
fill(h, h + N + 1, 0);	//9d	bfs.push(source);	//05	newedge.cap = cap;	//a9
h[source] = INF;	//51	ok[source] = 1;	//84	adjList[u].push_back(newedge);	//62
	//d3	dist[source] = 0;	//e8		//f0
for(int i = 0; i < edgeListSize; i++) {	//02		//22	newedge.to = u;	//8a
int w = Ev[i];	//9e	while(!bfs.empty()) {	//d4	newedge.rev = uPtr;	//c4
int idx = Ei[i];	//57		//e7	newedge.cap = cap;	//9e
int v = adjList[w][idx].to;	//4e	int v = bfs.front();	//4d	adjList[v].push_back(newedge);	//0c
p[i] = min(adjList[w][idx].cap,	//3c	bfs.pop();	//d5	}	//5b
h[w]);	//a2		//d5		//a5
h[v] += p[i];	//b3	for(int i = 0; i < adjList[v].size(); i++) {	//f4	int main() {	//fe
}	//53	if(adjList[v][i].cap == 0)	//5f		//4f
	//cd	continue;	//83	scanf("%d %d", &N, &M);	//28
if(h[sink] == 0)	//b7	int u = adjList[v][i].to;	//d9		//32
return 0;	//48	if(ok[u]) {	//6f	for(int i = 1; i <= M; i++) {	//af
	//b5	if(dist[u] == dist[v] + 1) {	//3f	int u, v, w;	//1e
fill(l, l + N + 1, 0);	//f9	Ev[bfsPtr] = v;	//7f	scanf("%d %d %d", &u, &v, &w);	//d3
l[sink] = h[sink];	//38	Ei[bfsPtr] = i;	//99	addEdge(u, v, w);	//9b
	//15	bfsPtr++;	//d1	}	//09
for(int i = edgeListSize - 1; i >= 0; i--) {	//6e	}	//e3		//b4
int w = Ev[i];	//c0	continue;	//ce	printf("%lld\n", FordFulkerson());	//22
int idx = Ei[i];	//76	}	//1f		//d9
int v = adjList[w][idx].to;	//ad	ok[u] = 1;	//31	return 0;	//a8
p[i] = min(p[i], min(h[w] - l[w], l[v]));	//a5	dist[u] = dist[v] + 1;	//7c	}	//ed
l[v] -= p[i];	//81	bfs.push(u);	//31		
l[w] += p[i];	//55	Ev[bfsPtr] = v;	//7c	6.11. Toposort.	
}	//de	Ei[bfsPtr] = i;	//1f	#include <queue>	//fc
	//b6	bfsPtr++;	//c7	#include <stack>	//c0
fill(h, h + N + 1, 0);	//01	}	//11	#include <vector>	//68
h[source] =	//37	}	//8e		//a5
l[source];	//4e		//65	using namespace std;	//c3
for(int i = 0; i < edgeListSize; i++) {	//74	return bfsPtr;	//9d		//5b
int w = Ev[i];	//cf	}	//9a	/// Input must be a DAG	//95
int idx = Ei[i];	//ed	11 FordFulkerson() {	//d8		//0a
int v = adjList[w][idx].to;	//56		//60	void toposort1(vector<vector<size_t>>& adj,	//4a
int rev_idx =	//d9	11 flow = 0;	//6f	vector<bool>& visited, size_t node,	//2c
adjList[w][idx].rev;	//66		//4b	stack<size_t>& ans) {	//50
p[i] = min(p[i], h[w]);	//df	while(true) {	//b0	visited[node] = true;	//ae
h[w] -= p[i];	//f9	11 augmenting = TidalFlow(BFS());	//bb		//44
h[v] += p[i];	//d1	if(augmenting == 0)	//0a	for(auto v : adj[node])	//45
adjList[w][idx].cap -=	//21	break;	//8f	if(!visited[v])	//13
p[i];	//ea	else	//5c	toposort1(adj, visited, v, ans);	//44
adjList[v][rev_idx].cap +=	//51	flow += augmenting;	//0c		//c6
p[i];	//94	}	//bd	ans.push(node);	//2f
}	//df		//5e	}	//4a
	//fe	return flow;	//f0		//c1
return h[sink];	//ba	}	//fc	void toposort2(vector<vector<size_t>>& adj,	//9d
}	//e3		//d5	vector<size_t>& in_degree, size_t n,	//3b
	//2e		//ee	vector<size_t>& ans) {	//89

<pre>queue<size_t> q; vector<bool> visited(adj.size()); size_t v; for(size_t i = 0; i < n; ++i) if(!in_degree[i]) q.push(i); while(!q.empty()) { v = q.front(), q.pop(); ans.push_back(v); for(auto u : adj[v]) if(visited[u] --in_degree[u]) continue; else q.push(u), visited[u] = true; } 6.12. Tsp. #include <algorithm> #define INF 2000000000 using namespace std; template <typename C> C tsp(size_t pos, size_t source, vector<vector<C>>& dist, size_t bitmask = 0) { if(bitmask == (1 << dist.size()) - 1) return dist[pos][source]; C tsp_cost = INF; for(size_t next = 0; next < dist.size(); ++next) { if(next != pos && !(bitmask & (1 << next)) && dist[pos][next] < INF) { C next_tsp_cost = tsp(next, source, dist, bitmask (1 << next)); if(next_tsp_cost < INF) tsp_cost = min(tsp_cost, dist[pos][next] + next_tsp_cost); } } return tsp_cost; }</pre>	<pre>//10 //6c //25 //90 //b7 //85 //50 //ba //ce //c9 //32 //92 //22 //18 //b8 //1f //36 //99 //a3 //5e //1e //7d //49 //c2 //25 //21 //53 //3f //f0 //45 //38 //0e //46 //56 //33 //ff //04 //ae //45 //67 //0f //3a //32 //21 //84 //16 //11 //b2 //f2 //22 //a7</pre>	<pre>bool leaf = false; int p = -1; char pch; int link = -1; int go[K]; int ind; int exitlink = -2; Vertex(int p = -1, char ch = '\$') : p(p), pch(ch) { fill(begin(next), end(next), -1); fill(begin(go), end(go), -1); } vector<Vertex> t(1); void add_string(string const& s, int num) { int v = 0; for(char ch : s) { int c = ch - 'a'; if(t[v].next[c] == -1) { t[v].next[c] = t.size(); t.emplace_back(v, ch); } v = t[v].next[c]; } t[v].leaf = true; t[v].ind = num; } int go(int v, char ch); int get_link(int v) { if(t[v].link == -1) { if(v == 0 t[v].p == 0) t[v].link = 0; else t[v].link = go(get_link(t[v].p), t[v].pch); } return t[v].link; } int get_exitlink(int v) { if(t[v].exitlink == -2) { if(v == 0) t[v].exitlink = -1; else { int pos = get_link(v); if(t[pos].leaf) t[v].exitlink = t[v].link; else { t[v].exitlink = get_exitlink(pos); } } } return t[v].exitlink; }</pre>	<pre>//00 //bb //14 //65 //dc //f9 //83 //16 //61 //50 //3f //56 //24 //21 //3a //47 //3e //1f //e1 //d8 //a1 //98 //42 //e0 //81 //05 //34 //3b //d6 //ed //f8 //18 //bc //1f //30 //d9 //95 //e8 //9e //1d //4e //28 //fd //ca //e6 //33 //65 //0d //dc //ea //2d //8e //91 //ad //f0 //12 int go(int v, char ch) { int c = ch - 'a'; if(t[v].go[c] == -1) { if(t[v].next[c] != -1) t[v].go[c] = t[v].next[c]; else t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch); } return t[v].go[c]; } 7.2. Aho Corasick. #include <bits/stdc++.h> using namespace std; template <typename T> class aho_corasick { public: template <typename U> class aho_node { friend class aho_corasick; aho_node<U>* parent; aho_node<U>* fall; unordered_map<T, aho_node<U>*> f; int memo = -1; aho_node() {} aho_node(aho_node<U>* parent, aho_node<U>* fall) : parent(parent), fall(fall) {} public: aho_node<U>* next_output = nullptr; bool ending = false; }; using node = aho_node<T>; private: node* root = new node(); vector<node*> to_reset; int get_output_count(node* n) { if(!n) return 0; if(n->memo != -1) return n->memo; to_reset.push_back(n); return n->memo = n->ending + get_output_count(n->next_output); } public: node* insert(const vector<T>& s) { node* curr = root; for(T c : s) { node* next = curr->f[c]; if(!next)</pre>	<pre>//e9 //b3 //21 //dd //de //76 //3d //31 //8c //70 //0c //66 //84 //8e //ca //b2 //ab //bc //11 //77 //4d //aa //4d //2e //e8 //37 //c6 //e7 //fc //17 //0f //57 //ea //b0 //cf //08 //5d //ac //58 //aa //46 //d4 //12 //9d //32 //35 //61 //23 //f6 //cf //14 //97 //90 //d3</pre>
7.1. Aho Corasick. <pre>#include <bits/stdc++.h> using namespace std; const int K = 26; struct Vertex { int next[K];</pre>	<pre>//84 //16 //11 //b2 //f2 //22 //a7</pre>	<pre>int pos = get_link(v); if(t[pos].leaf) t[v].exitlink = t[v].link; else { t[v].exitlink = get_exitlink(pos); } } return t[v].exitlink;</pre>	<pre>//12</pre>	

```

    next = curr->f[c] = new node(curr, root);
    curr = next;
}
curr->ending = true;
return curr;
}

void setup() {
    queue<pair<T, node*>> q;
    q.push({T(), root});
    root->parent = root;
    root->fall = nullptr;
    root->memo = 0;
    while(!q.empty()) {
        auto front = q.front();
        q.pop();
        T c = front.first;
        node* n = front.second;
        for(auto& p : n->f)
            q.push(p);
        if(n->parent != root) {
            node* curr = n->parent->fall;
            while(curr && !curr->f.count(c))
                curr = curr->fall;
            if(curr) {
                n->fall = curr->f[c];
                n->next_output = n->fall->ending
                    ? n->fall
                    : n->fall->next_output;
            }
        }
    }
}

void reset_memo() {
    for(node* n : to_reset)
        n->memo = -1;
    to_reset.clear();
}

pair<int, node*> query(const vector<T>& s) {
    int res = 0;
    node* curr = root;
    for(T c : s) {
        while(curr && !curr->f.count(c))
            curr = curr->fall;
        curr = curr ? curr->f[c] : root;
        res += get_output_count(curr);
    }
    return {res, curr};
}
};
```

7.3. Cyk.

```
#include <bits/stdc++.h>
```

```

using namespace std;

bool cyk(const vector<pair<char, string>>& G, const char S,
        const string& w) {
    auto n = w.size();
    auto table = vector<vector<unordered_set<char>>>(n);
    for(auto& r : table) {
        r = vector<unordered_set<char>>(n + 1,
            unordered_set<char>());
    }
    for(int i = 0; i < n; ++i) {
        for(auto& p : G) {
            auto& a = p.second;
            if(a.size() == 1 && a[0] == w[i])
                table[i][i + 1].insert(p.first);
        }
    }
    for(int l = 2; l <= n; ++l) {
        for(int i = 0; i < n - l + 1; ++i) {
            auto j = i + l;
            for(int k = i + 1; k < j; ++k) {
                for(auto& p : G) {
                    auto& a = p.second;
                    if(a.size() != 2)
                        continue;
                    if(table[i][k].count(a[0]) &&
                       table[k][j].count(a[1]))
                        table[i][j].insert(p.first);
                }
            }
        }
    }
    return (table[0][n].count(S));
}

7.4. Dbf.

#include <bits/stdc++.h>
using namespace std;
using u64 = uint64_t;
using i64 = int64_t;

struct hash_pair {
    size_t operator()(const pair<u64, u64>& p) const {
        return hash<u64>{}((p.first << 32) | p.second);
    }
};

vector<vector<u64>> DBF(const string& s) {
    size_t n = s.size();

    size_t ld = size_t(ceil(log2(n))) + 2;
    vector<vector<u64>> dbf(ld, vector<u64>(n + 1));
    vector<array<u64, 2>> term(n + 1);
```

```

    vector<u64> order(n + 1);
    vector<size_t> bucket(n + 1);
    vector<size_t> bucket_size(n + 1);
    vector<size_t> prefix(n + 1);
    unordered_set<char> chars;
    for(size_t i = 0; i < n; i++) {
        chars.insert(s[i]);
    }
    chars.insert('\0');
    vector<char> ordered;
    for(char c : chars) {
        ordered.push_back(c);
    }
    sort(ordered.begin(), ordered.end());
    unordered_map<char, size_t> mapper;
    for(size_t i = 0; i < ordered.size(); i++) {
        mapper[ordered[i]] = i;
    }
    for(size_t i = 0; i < n; i++) {
        dbf[0][i] = mapper[s[i]];
    }

    auto radix_sort = [&](int k) {
        fill(bucket_size.begin(), bucket_size.end(), 0);
        for(size_t j = 0; j < n + 1; j++) {
            bucket_size[term[order[j]][k]]++;
        }
        size_t partial = 0;
        for(size_t j = 0; j < n + 1; j++) {
            size_t tmp = bucket_size[j];
            bucket_size[j] = partial;
            partial += tmp;
        }
        for(size_t j = 0; j < n + 1; j++) {
            bucket[term[order[j]][k]]++ = order[j];
        }
        order = bucket;
    };

    for(size_t i = 1; i < dbf.size(); i++) {
        for(size_t j = 0; j < n + 1; j++) {
            term[j][0] = dbf[i - 1][j];
            term[j][1] =
                dbf[i - 1][j + (1 << (i - 1))] % (n + 1);
        }
        iota(order.begin(), order.end(), 0);

        radix_sort(1);
        radix_sort(0);
        dbf[i][order[0]] = 0;
        for(size_t j = 1; j < n + 1; j++) {
            auto o = order[j - 1];
            dbf[i][order[j]] =
                dbf[i][o] + (term[order[j]] != term[o]);
        }
    }
}
```

		<pre>//c3 int resfix, resmul, v; //d1 if(AN % 3 == 1) { //25 resfix = 1; resmul = RN >> 1; } else { resfix = 2; resmul = RN + 1 >> 1; } for(int i = v = 0; i < RN; i++) { v += i && (A[R[i - 1] + 0] != A[R[i] + 0] A[R[i - 1] + 1] != A[R[i] + 1] A[R[i - 1] + 2] != A[R[i] + 2]); SUBA[R[i] / 3 + (R[i] % 3 == resfix) * resmul] = v; } if(v + 1 != RN) { suffix_array(SUBA, RN); SA[0] = AN; for(int i = 1; i <= RN; i++) { SA[i] = SA[i] < resmul ? 3 * SA[i] + (resfix == 1 ? 2 : 1) : 3 * (SA[i] - resmul) + resfix; } } else { SA[0] = AN; memcpy(SA + 1, R, sizeof(int) * RN); } int NMN = RN; for(int i = RN = 0; i <= NMN; i++) { if(SA[i] % 3 == 1) { SUBA[RN++] = SA[i] - 1; } } radix_pass(A, AN, SUBA, RN, R); for(int i = 0; i <= NMN; i++) { SUBA[SA[i]] = i; } int ii = RN - 1; int jj = NMN; int pos; for(pos = AN; ii >= 0; pos--) { int i = R[ii]; int j = SA[jj]; int v = A[i] - A[j]; if(!v) { if(j % 3 == 1) { v = SUBA[i + 1] - SUBA[j + 1]; } else { v = A[i + 1] - A[j + 1]; } if(!v) { v = SUBA[i + 2] - SUBA[j + 2]; } } } } }</pre>	<pre>//b0 SA[pos] = v < 0 ? SA[jj--] : R[ii--]; //a8 } //bc } //e4 //58 char s[MAXN + 1]; //3f //92 /* Copies the string in s into A and reduces the characters //0f * as needed. */ //45 void prep_string() { //c3 int v = AN = 0; //4d memset(cnt, 0, 256 * sizeof(int)); //08 for(char* ss = s; *ss; ++ss, ++AN) //27 cnt[*ss]++; //3c for(int i = 0; i < AN; i++) //0d cnt[s[i]]++; //68 for(int i = 0; i < 256; i++) //80 cnt[i] = cnt[i] ? v++ : -1; //cd for(int i = 0; i < AN; i++) //8c A[i] = cnt[s[i]]; //52 } //71 //11 //d9 //0b //81 //e4 //4c //db //89 //ac //96 //d2 //13 //5e //f6 //e5 //c3 //11 //39 //71 //5d //51 //58 //b8 //0e //29 //80 //77 //28 //c7 //3c //4c //1d //7d //84 //48</pre>	<pre>//ed SA[pos] = v < 0 ? SA[jj--] : R[ii--]; //e9 } //ab } //63 //a8 char s[MAXN + 1]; //be //89 /* Copies the string in s into A and reduces the characters //7f * as needed. */ //35 void prep_string() { //9d int v = AN = 0; //19 memset(cnt, 0, 256 * sizeof(int)); //42 for(char* ss = s; *ss; ++ss, ++AN) //8d cnt[*ss]++; //7e for(int i = 0; i < AN; i++) //97 cnt[s[i]]++; //8b for(int i = 0; i < 256; i++) //17 cnt[i] = cnt[i] ? v++ : -1; //68 for(int i = 0; i < AN; i++) //cf A[i] = cnt[s[i]]; //ec } 7.6. Rabin Karp. struct hasher { int b = 311, m; vi h, p; hasher(string s, int _m) : m(_m), h(size(s) + 1), p(size(s) + 1) { p[0] = 1; h[0] = 0; rep(i, 0, size(s)) p[i + 1] = (ll)p[i] * b % m; rep(i, 0, size(s)) h[i + 1] = ((ll)h[i] * b + s[i]) % m; } int hash(int l, int r) { return (h[r + 1] + m - (ll)h[l] * p[r - l + 1] % m) % m; } }; 7.7. Suffix Array. struct entry { ii nr; int p; }; bool operator<(const entry& a, const entry& b) { return a.nr < b.nr; } struct suffix_array { string s; int n; vvi P; vector<entry> L; vi idx; suffix_array(string _s) : s(_s), n(size(s)) { L = vector<entry>(n), P.push_back(vi(n)), idx = vi(n); rep(i, 0, n) P[0][i] = s[i]; for(int stp = 1, cnt = 1; cnt >> 1 < n;</pre>
--	--	---	---	--

```

    stp++, cnt <= 1) { //70
P.push_back(vi(n)); //f3
rep(i, 0, n) L[L[i].p = i].nr = //21
    ii(P[stp - 1][i], //0b
    i + cnt < n ? P[stp - 1][i + cnt] : -1); //29
sort(L.begin(), L.end()); //4b
rep(i, 0, n) P[stp][L[i].p] = //02
    i > 0 && L[i].nr == L[i - 1].nr //0f
    ? P[stp][L[i - 1].p] //02
    : i; //92
} //f4
rep(i, 0, n) idx[P[size(P) - 1][i]] = i; //81
} //5c
int lcp(int x, int y) { //54
    int res = 0; //d2
    if(x == y) //36
        return n - x; //c3
    for(int k = size(P) - 1; k >= 0 && x < n && y < n; k--) //da
        if(P[k][x] == P[k][y]) //1a
            x += 1 << k, y += 1 << k, res += 1 << k; //bc
    return res; //03
} //21
}; //92

const Vector2D<T>& b) { //77
    return abs(a.y - b.y) > T(EPS) ? a.y < b.y : a.x < b.x; //dd
}; //22
auto cur = set<Vector2D<T>, decltype(cmpy)>(cmpy); //42
T mn = INF<T>; //44
pair<Vector2D<T>, Vector2D<T>> ans; //0a
for(int i = 0, l = 0; i < pts.size(); ++i) { //ff
    while(pts[i].x - pts[l].x > mn) //59
        cur.erase(pts[l++]); //f0
    auto it = cur.lower_bound({-INF<T>, pts[i].y - mn}); //f2
    auto jt = cur.upper_bound({INF<T>, pts[i].y + mn}); //26
    while(it != jt) { //f5
        auto dist = (*it - pts[i]).norm(); //09
        if(dist < mn) { //e2
            mn = dist; //5b
            ans = {*it, pts[i]}; //a6
        } //b0
        ++it; //79
    } //8b
    cur.insert(pts[i]); //75
} //70
return ans; //0e
} //6b

using namespace std; //ca
//ca
template <typename T> //b2
constexpr T INF = numeric_limits<T>::max(); //23
constexpr double EPS = 1e-6; //de
//9a
template <typename T> class Vector2D { //06
public: //57
    T x, y; //52
    Vector2D() { x = y = 0; } //7d
    Vector2D(T _x, T _y) : x(_x), y(_y) {} //c2
    T norm() const { return hypot(x, y); } //e8
    void normalize() { //ab
        T d = norm(); //fa
        x /= d; //e3
        y /= d; //a0
    } //50
    T operator*(const Vector2D<T>& o) const { //fb
        return x * o.x + y * o.y; //91
    } //ca
    Vector2D<T> operator*(const T& o) const { //6d
        return {x * o, y * o}; //9e
    } //9d
    Vector2D<T> operator/(const T& o) const { //9a
        return {x / o, y / o}; //75
    } //58
    T operator^(const Vector2D<T>& o) const { //b0
        return x * o.y - y * o.x; //63
    } //96
    Vector2D<T> normalized() const { //96
        auto v = *this; //84
        v.normalize(); //8e
        return v; //84
    } //8e
    Vector2D<T> operator+(const Vector2D<T>& o) const { //84
        return {x + o.x, y + o.y}; //8e
    } //8e
    Vector2D<T> operator-(const Vector2D<T>& o) const { //84
        return {x - o.x, y - o.y}; //8e
    } //8e
    T phase() const { return atan2(y, x); } //84
    Vector2D<T> rotated(T rad) { //84
        return {x * cos(rad) - y * sin(rad), //8e
                y * cos(rad) + x * sin(rad)}; //8e
    } //8e
}; //8e
```

7.8. Z Algorithm.

```

#include <bits/stdc++.h> //84
using namespace std; //8e
vector<size_t> z_values(const string& s) { //6f
    size_t n = s.size(); //23
    vector<size_t> z(n); //de
    size_t l = 0, r = 0; //9a
    z[0] = n; //06
    for(size_t i = 1; i < n; ++i) { //57
        if(i <= r) //52
            z[i] = min(z[i - l], r - i + 1); //7d
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) //c2
            ++z[i]; //e8
        if(i + z[i] - 1 > r) //ab
            l = i, r = i + z[i] - 1; //fa
    } //e3
    return z; //a0
} //50
}; //fb
```

8. COMPUTATIONAL GEOMETRY

8.1. Closest Pair.

```

#include "geocomp.hpp" //6f
template <typename T> //23
pair<Vector2D<T>, Vector2D<T>> //de
closest_pair(vector<Vector2D<T>> pts) { //96
    sort(pts.begin(), pts.end(), [](auto& a, auto& b) { //a6
        return abs(a.x - b.x) > T(EPS) ? a.x < b.x : a.y < b.y; //93
    }); //ce
    auto cmpy = [](const Vector2D<T>& a, //ea
    const Vector2D<T>& b) { //77
        return abs(a.y - b.y) > T(EPS) ? a.y < b.y : a.x < b.x; //dd
    }; //22
    auto cur = set<Vector2D<T>, decltype(cmpy)>(cmpy); //42
    T mn = INF<T>; //44
    pair<Vector2D<T>, Vector2D<T>> ans; //ff
    for(int i = 0, l = 0; i < pts.size(); ++i) { //59
        while(pts[i].x - pts[l].x > mn) //f0
            cur.erase(pts[l++]); //f2
        auto it = cur.lower_bound({-INF<T>, pts[i].y - mn}); //26
        auto jt = cur.upper_bound({INF<T>, pts[i].y + mn}); //f5
        while(it != jt) { //09
            auto dist = (*it - pts[i]).norm(); //e2
            if(dist < mn) { //5b
                mn = dist; //a6
                ans = {*it, pts[i]}; //b0
            } //79
            ++it; //8b
        } //75
        cur.insert(pts[i]); //70
    } //0e
    return ans; //6b
} //6b
```

8.2. Convex Hull.

```

#include "geocomp.hpp" //6f
template <typename T> //23
vector<Vector2D<T>> //de
convex_hull(vector<Vector2D<T>> points, //9a
            bool keep_aligned_points = false) { //06
    sort(begin(points), end(points), [](auto& a, auto& b) { //57
        return tie(a.x, a.y) < tie(b.x, b.y); //52
    }); //7d
    vector<Vector2D<T>> hull; //c2
    hull.reserve(points.size() + 1); //e8
    for(int phase = 0; phase < 2; ++phase) { //ab
        auto start = hull.size(); //fa
        for(auto& point : points) { //e3
            while(hull.size() >= start + 2 && //50
                ccw(hull.back(), point, hull[hull.size() - 2], //fb
                    keep_aligned_points)) //91
                hull.pop_back(); //ca
            hull.push_back(point); //6d
        } //9e
        hull.pop_back(); //9d
        reverse(begin(points), end(points)); //9a
    } //75
    if(hull.size() == 2 && hull[0] == hull[1]) //58
        hull.pop_back(); //b0
    return hull; //63
} //96
```

8.3. Geocomp.

```

#include <bits/stdc++.h> //84
using namespace std; //8e
```

```

using namespace std; //ca
//ca
template <typename T> //b2
constexpr T INF = numeric_limits<T>::max(); //23
constexpr double EPS = 1e-6; //de
//9a
template <typename T> class Vector2D { //06
public: //57
    T x, y; //52
    Vector2D() { x = y = 0; } //7d
    Vector2D(T _x, T _y) : x(_x), y(_y) {} //c2
    T norm() const { return hypot(x, y); } //e8
    void normalize() { //ab
        T d = norm(); //fa
        x /= d; //e3
        y /= d; //a0
    } //50
    T operator*(const Vector2D<T>& o) const { //fb
        return x * o.x + y * o.y; //91
    } //ca
    Vector2D<T> operator*(const T& o) const { //6d
        return {x * o, y * o}; //9e
    } //9d
    Vector2D<T> operator/(const T& o) const { //9a
        return {x / o, y / o}; //75
    } //58
    T operator^(const Vector2D<T>& o) const { //b0
        return x * o.y - y * o.x; //63
    } //96
    Vector2D<T> normalized() const { //96
        auto v = *this; //84
        v.normalize(); //8e
        return v; //84
    } //8e
    Vector2D<T> operator+(const Vector2D<T>& o) const { //84
        return {x + o.x, y + o.y}; //8e
    } //8e
    Vector2D<T> operator-(const Vector2D<T>& o) const { //84
        return {x - o.x, y - o.y}; //8e
    } //8e
    T phase() const { return atan2(y, x); } //84
    Vector2D<T> rotated(T rad) { //84
        return {x * cos(rad) - y * sin(rad), //8e
                y * cos(rad) + x * sin(rad)}; //8e
    } //8e
}; //8e
```

<pre> x * sin(rad) + y * cos(rad)); } void rotate(T rad) { *this = rotated(rad); } bool operator==(const Vector2D<T>& o) const { return (*this - o).norm() <= T(EPS); } bool operator!=(const Vector2D<T>& o) const { return !(*this == o); } }; template <typename T> T distancepp(const Vector2D<T>& a, const Vector2D<T>& b) { return (a - b).norm(); } template <typename T> T distancepl(const Vector2D<T>& p, const Vector2D<T>& c0, const Vector2D<T>& d0) { auto p0 = p - c0; auto d0n = d0.normalized(); auto offset = p0 * d0n; auto proj = d0n * offset; return (p0 - proj).norm(); } template <typename T> T angle(const Vector2D<T>& a, const Vector2D<T>& b) { return acos((a * b) / (a.norm() * b.norm())); } template <typename T> bool ccw(const Vector2D<T>& o, const Vector2D<T>& a, const Vector2D<T>& b, bool strict = false) { auto v = (a - o) ^ (b - o); return strict ? (v > T(0)) : (v >= T(0)); } 8.4. Poi. #include "template_short.h" using T = db; // or ll const T EPS = 1e-9; // adjust as needed using P = pair<T,T>; using vP = V<P>; using Line = pair<P,P>; int sgn(T a) { return (a>EPS)-(a<-EPS); } T sq(T a) { return a*a; } bool close(const P& a, const P& b) { return sgn(a.f-b.f) == 0 && sgn(a.s-b.s) == 0; } T norm(const P& p) { return sq(p.f)+sq(p.s); } T abs(const P& p) { return sqrt(norm(p)); } T arg(const P& p) { return atan2(p.s,p.f); } P conj(const P& p) { return P(p.f,-p.s); } P perp(const P& p) { return P(-p.s,p.f); }</pre>	<pre>//8b P dir(T ang) { return P(cos(ang),sin(ang)); } //1e //6d P operator-(const P& l) { return P(-l.f,-l.s); } //df P operator+(const P& l, const P& r) { //b3 return P(l.f+r.f,l.s+r.s); } //9a P operator-(const P& l, const P& r) { //b4 return P(l.f-r.f,l.s-r.s); } //32 P operator*(const P& l, const T& r) { //f2 return P(l.f*r,l.s*r); } //41 P operator*(const T& l, const P& r) { return r*l; } //85 P operator/(const P& l, const T& r) { //f1 return P(l.f/r,l.s/r); } //23 P operator*(const P& l, const P& r) { //c3 return P(l.f*r.f-l.s*r.s,l.s*r.f+l.f*r.s); } //cd P operator/(const P& l, const P& r) { //41 return l*conj(r)/norm(r); } //31 P& operator+=(P& l, const P& r) { return l = l+r; } //c1 P& operator-=(P& l, const P& r) { return l = l-r; } //e7 P& operator*=(P& l, const T& r) { return l = l*r; } //72 P& operator/=(P& l, const T& r) { return l = l/r; } //d8 P& operator*=(P& l, const P& r) { return l = l*r; } //da P& operator/=(P& l, const P& r) { return l = l/r; } //6a P unit(const P& p) { return p/abs(p); } //db T dot(const P& a, const P& b) { return a.f*b.f+a.s*b.s; } //cd T dot(const P& p, const P& a, const P& b) { return dot(a-p,b-p); } //49 T cross(const P& a, const P& b) { return a.f*b.s-a.s*b.f; } //1e T cross(const P& p, const P& a, const P& b) { //b2 return cross(a-p,b-p); } //e4 //82 P reflect(const P& p, const Line& l) { //a6 P a = l.f, d = l.s-l.f; //c9 return a+conj((p-a)/d)*d; } //84 P foot(const P& p, const Line& l) { //7d return (p+reflect(p,l))/(T)2; } //c8 bool onSeg(const P& p, const Line& l) { //ed return sgn(cross(l.f,l.s,p)) == 0 && sgn(dot(p,l.f,l.s)) //14 //80 //d9 //79</pre>	<pre> angle(polygon[i] - p, polygon[i + 1] - p); } return fabs(fabs(angles_sum) / (2.0 * M_PI)) > T(EPS); } 8.6. Polygon Area. #include "geocomp.hpp" // An n-gon is represented by a vector P[0..n] of n+1 // consecutive points, where P[0] = P[n]. template <typename T /* floating type */> T polygon_area(vector<Vector2D<T>>& polygon) { T area = T(0); for(size_t i = 0; i < polygon.size(); ++i) { area += polygon[i] ^ polygon[i + 1]; } return T(fabs(area)) / T(2); } 8.7. SegIsect. #include "Point.h" vP lineIsect(const Line& a, const Line& b) { T a0 = cross(a.f, a.s, b.f), a1 = cross(a.f, a.s, b.s); if(a0 == a1) return a0 == 0 ? vP{b.f, b.s} : vP{}; return {(b.s * a0 - b.f * a1) / (a0 - a1)}; } vP strictIsect(const Line& a, const Line& b) { T a0 = cross(a.f, a.s, b.f), a1 = cross(a.f, a.s, b.s); T b0 = cross(b.f, b.s, a.f), b1 = cross(b.f, b.s, a.s); if(sgn(a0) * sgn(a1) < 0 && sgn(b0) * sgn(b1) < 0) return {(b.s * a0 - b.f * a1) / (a0 - a1)}; return {}; } vP segIsect(const Line& a, const Line& b) { vP v = strictIsect(a, b); if(sz(v)) return v; set<P> s; #define i(x, y) if(onSeg(x, y)) s.insert(x) i(a.f, b); i(a.s, b); i(b.f, a); i(b.s, a); #undef i return {all(s)}; }</pre>